

GSIP

Complete Technical Master Guide

Grounded Simulation-in-the-loop Platform

A project explanation and study book covering the architecture, algorithms, data structures, software stack, implementation rationale, advanced optimisation concepts, security, testing, real-world uses, limitations, drawbacks, and future research directions.

Primary source	GSIP - Architecture & Design Specification v2 (6 August 2026)
Build source	GSIP - Build Brief & Handoff
Scope	v1 preserved concepts + v2 required architecture + advanced mastery extensions
Evidence note	Actual source-repository status must be inventoried before claiming every v2 subsystem is implemented.

Designed as a mastery reference, not a marketing summary.

Contents at a glance

- Part I - Project identity and mental model
- Part II - The trust and epistemic foundation
- Part III - Framing, classification, sensitivity, and grounding
- Part IV - Scenario generation, simulation, and optimisation
- Part V - Software architecture and infrastructure
- Part VI - Testing and engineering discipline
- Part VII - How the project can be used
- Part VIII - Drawbacks, limitations, and failure modes
- Part IX - Advanced extensions for mastering the subject
- Part X - Project roadmap, best uses, and definition of success
- Part XI - Mastery reference
- Appendix A - Detailed data structures
- Appendix B - Technology choice matrix
- Appendix C - End-to-end pseudocode
- Appendix D - Deep questions and complete answers
- Appendix E - Implementation checklist
- Appendix F - Compact mental model

How to read this guide

This document is written as both a project explanation and a study book. It begins with the project philosophy, then explains the end-to-end architecture, then studies every major algorithm and technology individually, then returns to the question of why these pieces are combined. The later sections cover implementation, testing, security, use cases, drawbacks, advanced extensions, and a mastery glossary.

A critical status note: the source material says that an existing v1 optimisation prototype/repository exists and that v2 is the successor architecture. The handoff explicitly instructs the implementer to inventory the existing repository before changing code and to preserve working v1 components. Therefore this guide distinguishes between **preserved/existing v1 concepts**, **v2 required architecture**, and **recommended advanced extensions**. Without the actual source repository, it would be inaccurate to claim that every v2 component is already implemented.

Part I - Project identity and mental model

1. What GSIP is

GSIP stands for **Grounded Simulation-in-the-loop Platform**. It is a general-purpose decision and optimisation platform designed for problems where the answer depends on numerical models, constraints, competing objectives, and uncertain real-world inputs.

The defining architectural rule is that the language model is allowed to understand the user's language, frame the problem, retrieve a method, ask clarification questions, propose candidate scenarios, and explain results, but it is not allowed to silently invent the numerical facts that determine the answer. Outcome numbers must come from deterministic domain code. Important input values must carry provenance showing where they came from. If an important value exists only because the language model recalled or guessed it, the system can refuse to run.

This makes GSIP different from a normal chatbot. A normal chatbot can compress the entire process into "question -> model response." GSIP deliberately decomposes the process:

1. Understand what kind of problem the user is asking.
2. Determine what information that class of problem requires.
3. Measure which of those inputs actually control the result.
4. Establish where the important values came from and how uncertain they are.
5. Refuse or warn when important facts are not sufficiently grounded.
6. Generate candidate decisions.
7. Evaluate candidates with deterministic domain simulation code.
8. Search efficiently for strong solutions and trade-offs.
9. Propagate uncertainty and model fidelity.
10. Explain only what the evidence supports.

The product is therefore best understood as a **trust-controlled operating system for simulation-based decision making** rather than as "an AI that knows every domain."

Project basis: Architecture v2, pp. 1-4; Build Brief, §§1-5.

1.1 Simulation-in-the-loop

A simulation is a program that maps a representation of a system and a candidate action to predicted or calculated outcomes. In abstract form:

Outcome = simulate(State, Action, ModelParameters)

For an electromechanical pack, the state might contain RPM, winding resistance, magnetic flux, load, thermal conditions, and material properties. Candidate actions might change turns, wire gauge, cooling configuration, or setpoints. The simulator calculates quantities such as voltage, current, power, temperature, efficiency, and constraint violations.

For a supply-chain pack, state may include inventories, suppliers, lead times, demand, warehouse capacities, and costs. Actions can include reorder points, supplier allocations, transportation modes, and stock targets. The simulator returns total cost, service level, stockout rate, delay, and resilience metrics.

The phrase **in the loop** means the simulator is repeatedly called as part of an optimisation process. The optimiser proposes a candidate, the simulator evaluates it, the result updates the optimiser, and the optimiser chooses what to try next. This loop is what converts a language-level problem into a computational search.

1.2 Domain-general does not mean omniscient

The generic platform intentionally contains no complete knowledge of air pollution, generators, supply chains, portfolios, treatment schedules, or other application areas. That knowledge belongs in **domain packs** and

playbooks. The core contains reusable mechanisms: provenance, gating, workflow, simulation execution, optimisation, uncertainty handling, auditing, and reporting.

This is an important architectural distinction. "Domain-general" means the same core can support many domains if suitable packs exist. It does not mean one universal model can validly solve every problem. The architecture explicitly treats abstention and competence boundaries as first-class outputs.

2. Why the project exists: the failure mode it is designed to stop

The central failure mode is **correct computation on invented premises.**

Suppose a language model is asked to optimise air-pollution interventions. It knows the general shape of the problem and may generate plausible source locations, source intensities, wind assumptions, and diffusion parameters. These values can be fed into a perfectly deterministic simulator. The simulator will return consistent numbers. An optimiser can then search those numbers efficiently. A polished report may follow.

The result can still be wrong because the important inputs were fabricated.

This yields the key distinction:

- **Determinism protects arithmetic.**
- **Provenance protects the origin of the inputs.**
- **Fidelity protects the interpretation of the model.**

A reproducible wrong input produces a reproducible wrong answer. Therefore "same input -> same output" is necessary for auditability but not sufficient for truth.

2.1 The trust boundary

The project moves the trust boundary to the point before numerical simulation. Probabilistic language-model outputs can exist in the framing zone, but decision-critical values cannot silently cross into the numerical core unless their provenance is acceptable.

A useful security analogy is:

Probabilistic claims -> epistemic firewall/provenance gate -> deterministic numerical core

The provenance gate is not a conventional network firewall. It is a policy engine for knowledge quality.

3. V1 and V2: what changed and why

The source material describes v1 as having several correct instincts that must be preserved. In particular, all result quantities come from deterministic simulation code, scoring is deterministic, optimisation is sample-efficient, runs are reproducible, the architecture separates generic platform code from domain-specific packs, and evaluations can be parallelised.

V2 keeps this computational engine but adds a trust-and-framing shell. The main changes are:

- A problem classifier with multi-label outputs, calibrated confidence, and the ability to abstain.
- Curated, versioned playbooks describing how a class of problem should be framed.
- Parameter triage using priors and empirical sensitivity screening.
- A provenance model that prevents bare values from losing their origin.
- A provenance gate combining criticality and provenance into RUN, FLAG, or BLOCK decisions.
- Triage-driven clarification that asks first about important unknowns.
- A fidelity descriptor that prevents a toy model from being presented as validated prediction.
- Split exploit/explore scenario generation so AI priors do not fence the search.
- Uncertainty-aware outcomes and reporting.
- Confidence-calibrated explanation bound by provenance and model fidelity.

The project sources summarize the difference well: v1 makes sure the mathematics is performed by code; v2 makes sure the mathematics is not performed on fabricated ingredients.

3.1 What must be preserved from v1

The Build Brief treats the following as non-negotiable design inheritance:

- Numbers from code only.
- Hybrid Bayesian optimisation plus NSGA-II over a shared evaluation pool.
- Parallel evaluation using Ray.
- Reproducibility through run specifications and seeds.
- Deterministic scoring with no AI judge in the objective function.
- Append-only auditing/run ledger.
- Generic platform/domain-pack separation.
- Configuration as code, containerised development, and reproducible runs.
- Optimiser validation using synthetic benchmark families such as ZDT/DTLZ.

The exact implementation status of each item should be verified against the source repository before development.

4. Design goals and non-goals

The architecture defines six major goals.

4.1 Grounded truth

Every value that materially affects the decision should be traceable to a user, measurement, named source, or deterministic computation from grounded parents. "Grounded" means attributable and inspectable, not infallible.

4.2 Honest confidence

The strength of the report must reflect the strength of evidence. A precise-looking number from a toy, unvalidated model should not be described as a forecast. A validated model with grounded inputs can justify stronger language.

4.3 Wide applicability

The platform should support many problem classes through plugins and playbooks while clearly exposing where coverage ends.

4.4 Reproducibility

A run specification and seed should make scenarios and numbers reproducible. Parallel completion order must not change the semantic identity of the evaluated candidates.

4.5 Auditability

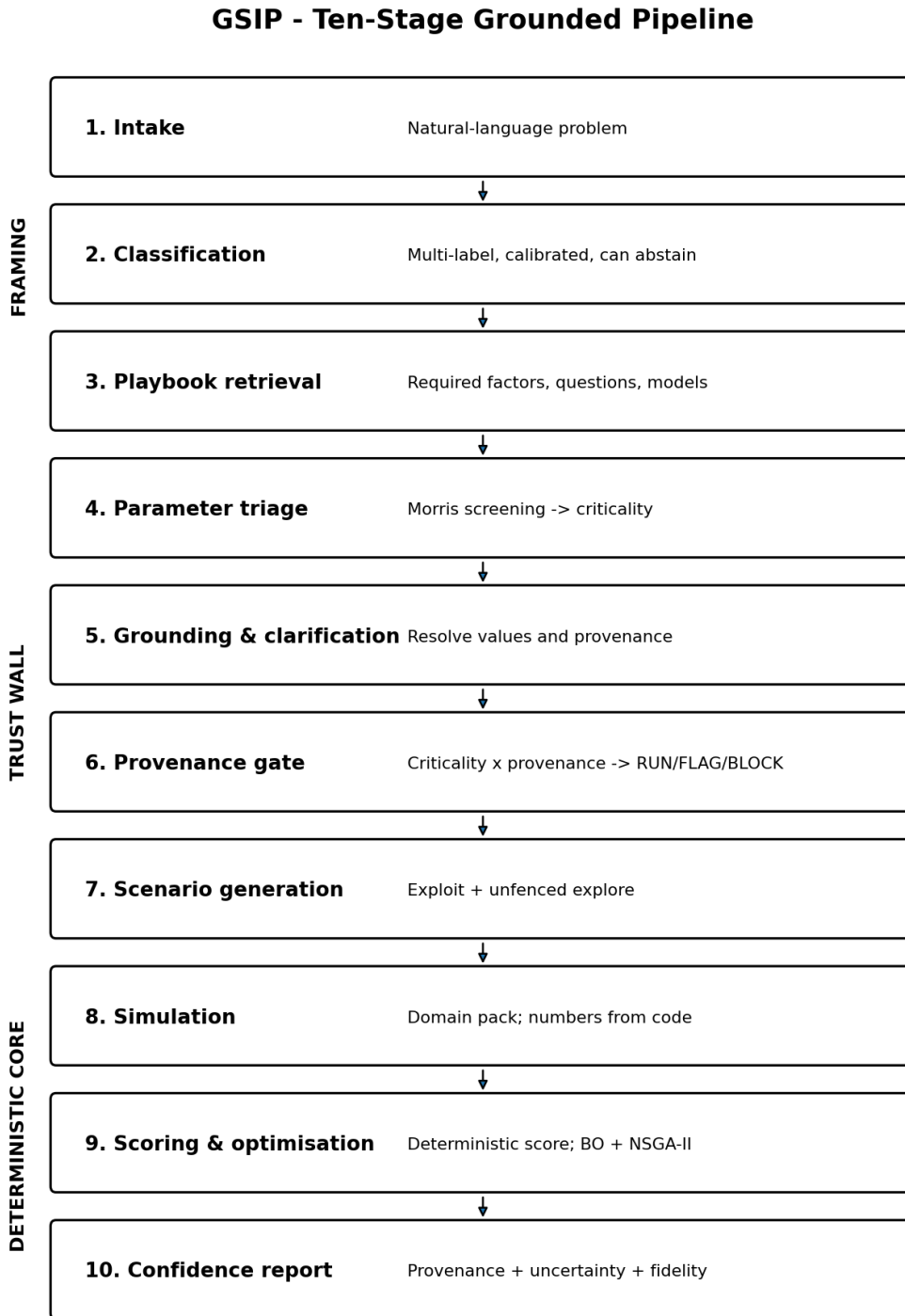
Every important decision, AI output, assumption, provenance tag, gate verdict, override, simulation result, and report state should be logged.

4.6 Discovery, not only confirmation

The optimiser must search beyond language-model priors. The exploration track intentionally samples the feasible space without being fenced by the AI's conventional suggestions.

The non-goals are equally important. GSIP is not a system that treats the language model as a numerical measuring device, not a claim of philosophical first-principles reasoning, and not a literal universal solver.

5. The ten-stage grounded pipeline



Run ledger surrounds every stage: decisions, assumptions, provenance, outputs, and overrides are recorded.

Figure 1. The ten-stage GSIP pipeline and trust boundary.

The architecture defines ten stages:

- 1. Intake** - receive a natural-language question.
- 2. Problem classification** - identify one or more problem frames, with calibrated confidence and abstention.

3. **Playbook retrieval** - retrieve required factors, characteristic questions, models, priors, pitfalls, and applicability bounds.
4. **Parameter triage** - determine which required factors actually control the answer.
5. **Grounding and clarification** - resolve values, attach provenance, retrieve evidence, or ask the user.
6. **Provenance gate** - combine criticality and provenance to decide RUN, FLAG, or BLOCK.
7. **Scenario generation** - combine grounded AI proposals with unfenced mathematical exploration.
8. **Simulation** - execute deterministic code in the user's domain pack and attach fidelity.
9. **Scoring and optimisation** - use deterministic scoring plus Bayesian optimisation and NSGA-II.
10. **Confidence-calibrated report** - explain results within the envelope justified by provenance, uncertainty, and model fidelity.

A run ledger surrounds all stages so the full process is reproducible and auditable.

5.1 Three jobs that must remain separate

The architecture emphasises three separate questions:

- **Classifier + playbook:** What do we need to know?
- **Triage:** Which of those variables actually matter in this case?
- **Provenance gate:** Where are those important values allowed to come from?

Collapsing these functions creates subtle failure modes. A classifier is not a sensitivity analyser. A sensitivity analyser cannot determine whether data is trustworthy. A provenance system cannot decide what variables the problem needs unless the framing exists first.

Part II - The trust and epistemic foundation

6. Large language models and small language models

An **LLM**, or large language model, is a neural sequence model trained to predict and generate language. At a simplified level, it estimates a conditional probability of the next token given the previous context:

$$P(\text{token_next} \mid \text{context})$$

Modern models learn rich internal representations and can perform classification, explanation, planning, synthesis, and tool selection. However, fluency and factual reliability are different properties. The model can generate an internally plausible statement without having an attributable external measurement behind it.

GSIP uses the LLM in roles where probabilistic language competence is valuable:

- interpreting natural-language requests;
- producing candidate problem frames;
- asking clarifying questions;
- proposing scenarios within grounded context;
- drafting playbooks for human ratification;
- writing readable explanations of deterministic results.

GSIP deliberately removes the LLM from roles where untraceable numerical invention would be dangerous:

- final numerical result generation;
- deterministic scoring;
- silent supply of decision-critical input facts.

An **SLM**, or small language model, is a smaller model that can be specialised for a narrow task such as problem classification. The Build Brief proposes a prompted general LLM for launch because it reduces development cost, then allows a fine-tuned SLM later behind the same `classify()` contract.

Why a specialised SLM can be attractive

A smaller classifier may provide lower latency, lower cost, local deployment, more stable outputs, and easier calibration. It also narrows the blast radius: the classifier only needs to map requests to problem types, not perform general conversation.

Advanced mastery: do not confuse model size with calibration

A smaller model is not automatically more trustworthy. Calibration must be measured on a representative held-out dataset, especially near decision thresholds and out-of-distribution examples. The model's role should be evaluated using metrics such as log loss, Brier score, calibration error, precision/recall by class, abstention coverage, and selective risk.

7. Determinism, reproducibility, and auditability

Determinism means identical inputs produce identical outputs. In a scientific or optimisation pipeline, complete determinism can require more than pure functions. It can require fixed random seeds, deterministic numerical libraries where possible, versioned software environments, fixed dataset snapshots, stable floating-point settings, and deterministic candidate IDs.

Reproducibility means another run can recreate the relevant computation from recorded information. A reproducible GSIP run should record at least:

- run specification;
- code and pack version;
- playbook version;
- model/provider identifiers for AI-generated framing;

- random seeds;
- exact grounded values and source identifiers;
- optimiser configuration and budget;
- simulation environment/container version;
- gate decisions and overrides.

Auditability goes further. It asks whether a reviewer can reconstruct **why** the system was allowed to proceed and what assumptions were made.

Determinism is not truth

If $x = 100$ is false and the deterministic simulator correctly computes $10 \times x = 1000$, the output is reproducible but wrong. This is why GSIP pairs determinism with provenance and fidelity.

Advanced mastery: floating-point reproducibility

Bit-for-bit reproducibility can be difficult across CPUs, GPUs, BLAS libraries, and distributed environments because floating-point addition is not associative. A practical project should distinguish:

- semantic reproducibility: same candidate decisions and materially same metrics;
- numerical tolerance reproducibility: outputs equal within defined tolerances;
- bitwise reproducibility: exact binary identity.

Not every domain needs the strictest level, but the level should be declared.

8. Provenance: definition, taxonomy, and provenance algebra

Data provenance is information describing where a value came from, how it was obtained, and how it was transformed. GSIP treats provenance as part of the value rather than as optional metadata stored elsewhere.

The central object is conceptually:

```
class ParameterValue:
    name: str
    value: Any
    provenance: Provenance
    source_id: str | None
    uncertainty: float | None
    parents: list[str]
    notes: str | None
```

This is a major design improvement over storing raw primitives such as `wind_speed = 8.4`.

8.1 Provenance classes

USER_SUPPLIED

The user explicitly states the value. It is attributable to the user. This does not guarantee the user is correct, but the claim is traceable.

MEASURED

The value originates from an instrument or sensor. Good implementations should also record device ID, calibration information, timestamp, units, measurement uncertainty, and possibly raw sample references.

RETRIEVED

The value comes from a named dataset, API, database, file, or tool. The crucial property is attribution. Retrieval is not equivalent to truth; it produces a checkable claim.

COMPUTED

The value is derived deterministically from parent values. A computed value inherits the weakest relevant provenance among its parents so that arithmetic cannot wash away a weak source.

DEFAULT

A pack supplies a documented assumption. This is ungrounded in the current case but explicit. Defaults should carry a source such as model documentation and a defensible uncertainty range.

LLM_RECALL

The language model supplied a value from its learned memory or contextual inference without an attributable external source. This is the floor of the trust taxonomy.

8.2 Provenance inheritance: preventing laundering

Suppose:

A = MEASURED

B = LLM_RECALL

and:

$C = A * B$

If C were simply labelled COMPUTED and treated as grounded, the system would have laundered B through arithmetic. GSIP therefore requires C to inherit the weakest parent provenance for gating purposes.

A more advanced implementation can model provenance as a directed acyclic graph (DAG). Each computed node references parent nodes, and the trust policy traverses ancestry. This supports explanations such as: "The output is uncertain primarily because coefficient C ultimately depends on a defaulted material property."

8.3 Advanced provenance features

For a mature system, consider adding:

- source hashes so the exact referenced file/version can be verified;
- timestamps and validity intervals;
- unit and dimensional metadata;
- transformation code/version identifiers;
- cryptographic signatures for trusted organisational sources;
- source-quality scores separate from provenance class;
- conflict records when two grounded sources disagree;
- lineage graphs exposed in the UI.

This leads to an important principle: **provenance is about traceability, while source quality is a separate evaluation dimension.**

9. The provenance gate

The **provenance gate** is the core policy mechanism that turns traceability into enforceable behaviour. It combines two dimensions:

1. **Criticality** - how strongly the parameter influences the decision.
2. **Provenance** - how the value was obtained.

The source architecture defines a matrix approximately as follows:

Criticality	Grounded	Declared Default	LLM Recall
Decision-critical	RUN	FLAG + user confirmation	BLOCK or explicit logged override

Criticality	Grounded	Declared Default	LLM Recall
Moderate	RUN	RUN + log	FLAG + log
Low	RUN	RUN	RUN + log

The central policy is simple: **a decision-critical parameter may never silently use an LLM-recalled value.**

9.1 RUN, FLAG, and BLOCK

RUN means the trust policy is satisfied.

FLAG means execution may continue, but the assumption must be visible and recorded. A flag can trigger user confirmation, widened uncertainty, or a stronger report caveat.

BLOCK means the workflow pauses. The system must ask the user, retrieve a source, obtain a measurement, or receive an explicit override.

9.2 Why the gate is better than a global "never trust AI" rule

Not every AI-generated token matters to the decision. If the model proposes a harmless label or low-sensitivity default, blocking the entire workflow would make the product unusable. GSIP therefore focuses strictness where sensitivity shows the output is actually vulnerable.

The deeper design principle is:

Required evidence strength increases with decision influence.

9.3 Overrides

An override is useful for exploratory analysis. A user may deliberately say, "Assume 0.8 so I can see the direction of effect." The system can allow the run if the override is explicit, logged, and propagated to the report. The result must not be silently upgraded from hypothetical to predictive.

9.4 Advanced mastery: gate policy as policy-as-code

The simple matrix can eventually become a versioned policy engine. Policies can consider:

- criticality;
- provenance;
- source quality;
- age/staleness;
- uncertainty;
- model fidelity;
- domain risk level;
- regulatory class;
- user role;
- whether the result is advisory or safety-critical.

The important engineering property is that policy decisions remain deterministic, testable, and logged.

10. Model fidelity, validation, and applicability

Provenance answers: "Where did the inputs come from?" Fidelity answers: "How faithful is the computational model to reality?" Both dimensions are necessary.

The architecture proposes fidelity tiers:

- **TOY** - conceptual or demonstrative model.
- **REDUCED_ORDER** - simplified model capturing dominant behaviour.
- **VALIDATED** - model supported by relevant validation evidence within declared bounds.

- **CALIBRATED** - model parameters adjusted against real observations and supported by calibration evidence.

It separately tracks validation status such as:

- UNVALIDATED;
- BACKTESTED;
- EXPERT_REVIEWED;
- GROUND_TRUTH_CALIBRATED.

This separation matters. A mathematically sophisticated model can still be unvalidated. A simpler model can be strongly backtested for a narrow use.

10.1 Applicability bounds

Every model is valid only under assumptions and ranges. Examples include temperature range, Reynolds number regime, voltage range, geographic scale, or lead-time assumptions. Applying a correct model outside its validity region is a model error, not a data-provenance error.

10.2 Known limitations

A pack should disclose simplifications such as ignored transients, omitted harmonics, simplified geometry, stationary demand assumptions, or missing feedback effects. These limitations must travel into the report.

10.3 Model validation versus calibration

Validation asks whether the model produces sufficiently accurate behaviour for intended use.

Calibration estimates parameters to align model outputs with observed data.

Calibration without independent validation risks overfitting. A mature workflow separates calibration data from validation data and records metrics appropriate to the domain.

10.4 Advanced model credibility

For higher-assurance use, the coarse tiers can be supplemented with:

- validation datasets and holdout results;
- uncertainty/error distributions;
- verification evidence that the code correctly solves its equations;
- sensitivity to discretisation and solver tolerances;
- model-card style documentation;
- independent review or certification;
- applicability tests that automatically reject out-of-range runs.

11. Uncertainty and confidence

Uncertainty is not one thing. A useful decomposition is:

- **Aleatoric uncertainty** - inherent variability/noise in the system or observations.
- **Epistemic uncertainty** - uncertainty due to limited knowledge, sparse data, uncertain parameters, or model form.
- **Numerical uncertainty** - discretisation, solver tolerances, finite precision.
- **Model-form uncertainty** - the chosen equations omit or simplify real mechanisms.

GSIP attaches uncertainty to input values and carries uncertainty in OutcomeBundle so that a final Pareto point is not presented as falsely exact.

11.1 Sensitivity-based propagation

If a parameter has uncertainty σ_i and local/global sensitivity magnitude S_i , a first-order approximation can use contributions related to $S_i * \sigma_i$. A variance-style approximation under independence can resemble:

$\text{Var}(Y) \text{ approximately } \sum((dY/dX_i)^2 * \text{Var}(X_i))$

The architecture uses the triage sensitivities as a cheap approximation instead of requiring a separate Monte Carlo pass for every run.

11.2 When this approximation can fail

First-order propagation can be inaccurate when:

- the model is highly nonlinear;
- inputs are strongly correlated;
- interactions dominate;
- uncertainty ranges are very wide;
- discontinuities or feasibility boundaries are present.

In those cases, higher-fidelity uncertainty methods can include Monte Carlo simulation, polynomial chaos, Gaussian-process emulators, bootstrap methods, or domain-specific uncertainty propagation.

11.3 Confidence is not a single probability

The report's confidence should not be confused with classifier confidence. There are multiple confidence concepts:

- classifier confidence about the problem frame;
- uncertainty in measured/retrieved inputs;
- uncertainty in simulated outcomes;
- validation evidence for the model;
- confidence that one Pareto solution dominates another;
- linguistic strength of the final explanation.

A mature UI should keep these distinct rather than displaying one magical "confidence 87%" number.

Part III - Framing, classification, sensitivity, and grounding

12. Problem classification

Problem classification maps a natural-language request to one or more problem types. A problem type is not merely an industry category; it is a reusable reasoning frame. For example, "reduce urban pollution" can involve spatial dispersion, policy feasibility, public-health impact, and economic allocation simultaneously.

GSIP therefore requires **multi-label classification** rather than forcing exactly one class.

A conceptual output is:

spatial/dispersion	0.82
policy/feasibility	0.55
public-health	0.41

These scores should be calibrated because downstream thresholds may decide whether to retrieve a playbook, ask for disambiguation, or abstain.

12.1 Multi-class versus multi-label

In **multi-class** classification, exactly one class is selected from mutually exclusive classes. In **multi-label** classification, several labels can simultaneously be true. GSIP needs the second because real decisions often occupy multiple frames.

12.2 Frame disambiguation

When highly ranked frames imply different objectives or required variables, the system should surface the conflict instead of silently picking one. This is particularly important for policy, health, and business problems where the meaning of "best" depends on stakeholder priorities.

12.3 Advanced classifier architectures

Possible implementations include:

- prompted LLM with structured JSON output;
- embedding retrieval followed by rule/model ranking;
- fine-tuned transformer classifier;
- hierarchical classifier where broad families route to specialised sub-classifiers;
- ensemble models combining lexical, embedding, and learned scores.

Whichever implementation is used, the public contract should remain stable: ranked tags, calibrated confidence, abstention state, and optional generic mode.

13. Probability calibration

A classifier can be accurate but badly calibrated. If a model says "0.95" frequently when it is correct only 70% of the time, confidence thresholds become dangerous.

13.1 Temperature scaling

Given class logits z_i , the usual softmax is:

$$p_i = \exp(z_i) / \sum_j \exp(z_j)$$

Temperature scaling introduces a learned scalar T :

$$p_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

$T > 1$ softens overconfident probabilities; $T < 1$ sharpens them. The temperature is fit on held-out calibration data, typically by minimising negative log-likelihood.

Temperature scaling is attractive because it is simple and often preserves ranking while improving confidence estimates.

13.2 Isotonic regression

Isotonic calibration learns a monotonic mapping from raw scores to empirical probabilities. It is more flexible than a single temperature but can overfit when calibration data is small.

13.3 Calibration metrics

Useful metrics include:

- **Brier score**: mean squared error of probabilistic predictions;
- **negative log-likelihood**: penalises confident wrong predictions strongly;
- **expected calibration error (ECE)**: compares confidence bins with empirical accuracy;
- **reliability diagrams**: visual calibration curves.

For GSIP, calibration should be evaluated not only globally but also near abstention thresholds, by problem family, and on shifted/OOD data.

14. Abstention, open-set recognition, and selective prediction

A domain-general front-end receives problems that may not match any validated frame. Therefore the classifier needs the ability to abstain.

Open-set recognition is the problem of recognising inputs that do not belong to known classes. This is harder than ordinary closed-set classification because neural models can remain confidently wrong far from the training distribution.

14.1 Selective prediction

In selective prediction, the model is allowed to answer only on a subset of examples. Let coverage be the fraction of requests it accepts and selective risk be the error rate on accepted requests. A system can trade coverage for reliability.

GSIP can use this principle operationally:

- high-confidence known frame -> proceed;
- ambiguous frame -> ask disambiguation;
- low-confidence or OOD -> abstain;
- optional generic mode -> proceed only with heightened flagging and narrow claims.

14.2 OOD signals

Possible signals include:

- low maximum calibrated probability;
- high entropy;
- distance from known embedding clusters;
- energy-based scores;
- ensemble disagreement;
- specialised outlier detectors;
- retrieval failure to find a compatible playbook/pack.

No single score solves open-set recognition. The architecture correctly treats this as a research risk.

15. Playbooks: reusable problem-solving structure

A **playbook** is a curated, versioned representation of how a problem class should be approached. It is not the simulator itself. It describes the **structure of inquiry**.

Core fields include:

- `required_factors` - variables the problem usually depends on;
- `characteristic_questions` - questions an expert asks;
- `candidate_models` - domain packs or governing models that can represent the problem;
- `sensitivity_priors` - initial beliefs about which factors are influential;
- `known_pitfalls` - known ways the problem can be misframed or misinterpreted;
- `applicability_bounds` - boundaries where the framing stops being valid.

15.1 Why playbooks must be curated

If the LLM is allowed to generate a new method unchecked for every request, hallucination simply moves from the data layer into the framing layer. The project therefore proposes LLM-draft -> human-ratify -> version-control.

15.2 Versioning and governance

A mature registry should store:

- semantic version;
- author and reviewers;
- date approved;
- references;
- supported pack versions;
- validation examples;
- known failure cases;
- deprecation status;
- next review date.

This turns playbooks into governed knowledge assets rather than transient prompts.

16. Parameter triage and sensitivity analysis

Triage asks: **Of everything the playbook says could matter, which values actually move the answer enough to deserve strict grounding?**

The architecture combines two sources:

1. **Prior sensitivity** from the playbook.
2. **Empirical sensitivity** measured by running the domain pack at its cheapest useful fidelity.

The output ranks each parameter as CRITICAL, MODERATE, or LOW and records the sensitivity source.

This ranking controls the provenance gate and the order in which clarification questions are asked.

17. Morris elementary-effects screening

The project chooses the **Morris method** as the default empirical screening algorithm because it provides useful global sensitivity screening with substantially fewer simulations than a full variance-based decomposition.

For a model $Y = f(X_1, \dots, X_k)$, the elementary effect for parameter i at a point x can be written conceptually as:

$$EE_i(x) = [f(x + \Delta e_i) - f(x)] / \Delta$$

where e_i changes only parameter i .

The method samples multiple trajectories through the feasible parameter space and computes multiple elementary effects for each factor.

Two common summaries are:

- μ^* - mean absolute elementary effect; large values indicate strong overall influence;
- σ - variation in elementary effects; large values can indicate nonlinearity or interactions.

17.1 Why Morris fits GSIP

The triage stage does not necessarily need a perfect decomposition of output variance. It needs a cost-effective answer to: "Which few parameters deserve expensive truth-finding?" Morris is well suited to this screening role.

17.2 Limitations

Morris results depend on parameter ranges, trajectory design, model noise, and chosen outputs. A parameter can appear unimportant if the tested range is too narrow. Discontinuities or constraints can distort effects. For multi-output problems, criticality may differ by objective.

A good implementation therefore records the tested ranges, number of trajectories, seed, output metrics, and aggregation rule.

18. Sobol variance-based sensitivity analysis

Sobol sensitivity analysis decomposes the variance of model output into contributions from input variables and their interactions. It is more quantitatively informative than Morris but generally more expensive.

For $Y = f(X_1, \dots, X_k)$, the first-order index for X_i is conceptually:

$$S_i = \text{Var}_{X_i}(E[Y | X_i]) / \text{Var}(Y)$$

It represents the fraction of output variance attributable to X_i alone. A total-order index includes the direct effect plus all interactions involving X_i .

18.1 Morris versus Sobol

Use Morris when the question is primarily screening: "Which variables matter?"

Use Sobol when the question is deeper decomposition: "How much variance is caused by each variable and interaction?"

For expensive simulators, a practical path is Morris first, then Sobol only on a reduced set of influential parameters or on a fast surrogate.

18.2 Do not confuse Sobol analysis with Sobol sequences

Sobol **sensitivity indices** decompose variance. Sobol **sequences** are low-discrepancy point sequences used for space-filling sampling. GSIP uses the name in both contexts for different purposes.

19. Grounding and clarification

Grounding converts required parameters into explicit `ParameterValue` objects with provenance and uncertainty.

Possible grounding routes include:

- ask the user;
- retrieve a named dataset or API;
- read a measurement source;
- compute from already grounded parents;
- accept a documented default if policy permits.

Clarification is the human-facing part of this process. The crucial v2 improvement is **triage-driven clarification**. The system asks first about values that are both decision-critical and ungrounded.

This prevents "clarification theatre" where a system asks easy questions while leaving the actual drivers as guesses.

19.1 A clarification question should declare what it resolves

A good internal object includes:

- the parameter being grounded;
- why it matters;
- the question;
- what provenance class the answer will produce;
- whether it is required;
- any declared default if the user skips.

This makes clarification machine-auditable rather than merely conversational.

20. Retrieval and the difference between traceability and truth

Retrieval can convert an untraceable claim into an attributable claim. If a value is retrieved from a file, API, instrument database, or named dataset, the system can store a source identifier.

However, a citation can still point to wrong or stale data. Therefore a mature grounding pipeline should consider source quality, freshness, scope, unit consistency, temporal/geographic applicability, and conflict resolution.

Advanced source-resolution pipeline

A stronger future grounding flow can be:

1. retrieve candidate sources;
2. verify schema/units;
3. filter by applicability and time range;
4. compare conflicting values;
5. attach source-quality metadata;
6. ask the user when ambiguity remains;
7. only then create the final ParameterValue.

This preserves the core principle: **retrieval provides attribution, not a guarantee of correctness.**

Part IV - Scenario generation, simulation, and optimisation

21. Scenario generation: exploit and explore

After the trust gate is satisfied, the system needs candidate actions/configurations to evaluate. The architecture uses a split budget:

- approximately 60% **exploit** candidates - grounded AI proposals informed by the playbook and real context;
- approximately 40% **explore** candidates - space-filling mathematical samples that are not fenced by AI priors.

The ratio is explicitly tunable. It is a launch heuristic, not a universal law.

21.1 Exploitation

Exploitation concentrates evaluations in regions that appear realistic or promising. The LLM can be valuable because it knows conventional designs and can translate context into plausible candidate structures.

21.2 Exploration

Exploration protects the optimiser from the language model's tendency to reproduce conventional patterns. It intentionally covers regions the AI would not suggest.

This is important because an optimisation system should be capable of discovery, not merely confirmation.

21.3 Grounding still applies to AI scenarios

Specific numerical values introduced by an AI-proposed scenario remain LLM_RECALL unless they are derived from grounded constraints or explicitly grounded. Scenario generation does not create a provenance exemption.

22. Latin Hypercube Sampling

Latin Hypercube Sampling (LHS) is a stratified sampling method for covering a multidimensional space efficiently.

For each input dimension, the range is divided into N equally probable intervals. One sample is taken from each interval, and the intervals across dimensions are combined using permutations. Each one-dimensional projection therefore covers the full range without the clumping common in naive random sampling.

LHS is useful for:

- initial design of experiments;
- exploration candidates;
- surrogate-model training sets;
- uncertainty studies.

22.1 Strengths and weaknesses

Strengths: simple, efficient, good marginal coverage, easy to constrain.

Weaknesses: good one-dimensional stratification does not automatically guarantee optimal high-dimensional space filling; designs may need maximin optimisation; hard constraints can invalidate points.

23. Sobol low-discrepancy sequences

A **Sobol sequence** is a quasi-random, low-discrepancy sequence designed to cover the unit hypercube more evenly than independent random points.

The goal is not randomness for its own sake. The goal is to minimise large empty regions and clustering. In moderate dimensions, low-discrepancy sequences can produce efficient space-filling designs for exploration.

To use Sobol points in GSIP:

1. generate points in $[0,1]^d$;
2. transform each dimension into its actual variable range or distribution;
3. map discrete/categorical dimensions carefully;
4. reject or repair infeasible points;
5. preserve deterministic sequence indices for reproducibility.

Advanced consideration: constrained feasible regions

If feasible configurations occupy a tiny irregular subset of the hypercube, naive rejection wastes samples. More advanced approaches include constraint-aware sampling, generative feasible-region models, repair operators, or sampling in a transformed parameterisation that guarantees feasibility.

24. Domain packs and the plugin contract

A **domain pack** is an executable plugin containing the domain-specific model while conforming to a fixed platform contract. This is the main mechanism that makes GSIP general across domains.

The preserved contract is conceptually:

```
class DomainPackBase:
    def state_schema(self): ...
    def action_schema(self): ...
    def simulate(self, state, action): ...
    def score(self, outcome): ...
    def feasibility(self, state, action): ...
    def cost_model(self, fidelity): ...
```

The v2 contract adds mandatory fidelity information and uncertainty-aware outputs.

24.1 state_schema

Defines initial conditions and fixed context. Strict schemas prevent the optimiser from running with malformed data and provide units/ranges for authoring tools.

24.2 action_schema

Defines decision variables and levers the optimiser is allowed to change. It describes types, ranges, categories, dependencies, and constraints.

24.3 simulate

Runs deterministic domain code and returns an OutcomeBundle. This is the numerical truth boundary: final metrics originate here, not from the LLM.

24.4 score

Transforms outcomes into deterministic objectives or ranking metrics. AI should not act as a subjective scoring oracle when the project claims reproducibility.

24.5 feasibility

Rejects impossible candidates. Without feasibility checks, optimisation often discovers mathematically attractive but physically, legally, or operationally impossible designs.

24.6 cost_model

Estimates the computational cost of a simulation at each fidelity. This supports budget-aware search and later multi-fidelity optimisation.

25. Deterministic simulation and numerical verification

A simulator can be deterministic and still be wrong due to coding errors. Therefore **verification** and **validation** must be separated:

- **Verification:** Did we implement/solve the intended model correctly?
- **Validation:** Is the intended model a sufficiently accurate representation of reality for this use?

Pack verification can include unit tests, analytic test cases, conservation checks, convergence tests, dimensional analysis, regression tests, and comparisons against trusted solvers.

Advanced mastery: numerical stability

Optimisers frequently push models to boundaries where numerical issues appear. Domain packs should handle:

- singularities;
- invalid logarithms/divisions;
- solver non-convergence;
- overflows;
- unstable time steps;
- infeasible geometry;
- discontinuities.

A failed simulation should return a structured failure/feasibility outcome rather than silently produce NaN values that poison the optimiser.

26. Deterministic scoring, objectives, and constraints

An optimisation problem can be formalised as:

minimise $f(x)$

subject to:

$g_i(x) \leq 0$

$h_j(x) = 0$

For multi-objective problems:

minimise $[f_1(x), f_2(x), \dots, f_m(x)]$

Examples include maximising generator power while minimising cost and temperature, or minimising supply-chain cost while maximising service and resilience.

26.1 Why not use an LLM judge

Language-model scoring can vary with wording, prompt context, provider, and stochastic sampling. Deterministic scoring makes objective definitions inspectable and reproducible.

An LLM may help a user *choose* objective definitions, but once chosen, the numerical scoring rule should be code.

27. Bayesian optimisation: complete conceptual foundation

Bayesian optimisation (BO) is a strategy for optimising expensive black-box functions. It is especially useful when each real evaluation requires a costly simulation, experiment, or external process.

The key idea is to avoid evaluating candidates blindly. BO maintains a cheap statistical **surrogate model** of the expensive objective and uses an **acquisition function** to decide where the next real evaluation is most valuable.

The loop is:

1. evaluate an initial design;
2. fit/update the surrogate;
3. compute an acquisition function over candidate space;
4. maximise the acquisition to select the next candidate or batch;
5. evaluate the real simulator;
6. add the result to the shared archive;
7. repeat until budget/convergence.

27.1 Surrogate models

A surrogate approximates $f(x)$ and should provide uncertainty as well as predicted value. Common choices include Gaussian processes, random forests, neural surrogates, and specialised models for mixed/discrete spaces.

The source Build Brief names BoTorch as the launch BO library, which naturally supports Gaussian-process based and modern acquisition workflows.

28. Gaussian processes

A **Gaussian process (GP)** defines a probability distribution over functions. Instead of predicting a single value at x , it provides a posterior distribution with mean and variance.

Conceptually:

$$f(x) \sim \text{Normal}(\mu(x), \sigma^2(x))$$

A GP is defined by a mean function and a covariance/kernel function. The kernel encodes beliefs about how function values relate as inputs move through the design space.

28.1 Kernels

Common kernels include squared-exponential/RBF and Matern kernels. A length-scale parameter controls how rapidly the function is expected to change in a dimension.

Automatic relevance determination (ARD) uses separate length scales per dimension, which can reveal which variables affect the surrogate strongly. However, ARD is not a substitute for proper sensitivity analysis and can become unreliable with sparse data.

28.2 GP strengths

- strong performance with small expensive datasets;
- principled uncertainty estimates under model assumptions;
- differentiable acquisition functions;
- natural fit for continuous low/moderate-dimensional spaces.

28.3 GP limitations

- classical exact GP training scales cubically with number of observations;
- high-dimensional spaces are difficult;
- categorical/mixed variables require specialised kernels or transformations;
- discontinuities and heteroscedastic noise can violate assumptions;
- poor kernels can produce misleading uncertainty.

29. Acquisition functions

The acquisition function determines where expensive evaluation is most valuable.

29.1 Expected Improvement (EI)

EI measures expected improvement over the best observed value while integrating surrogate uncertainty. It naturally balances exploitation and exploration because a point can have high EI either due to a strong predicted mean or due to uncertainty that leaves meaningful upside.

29.2 Probability of Improvement (PI)

PI asks for the probability that a candidate exceeds a target. It is simple but can become overly exploitative because it does not care how much improvement occurs.

29.3 Upper Confidence Bound (UCB)

For maximisation, a common form is:

$$\text{UCB}(x) = \mu(x) + \kappa * \sigma(x)$$

The kappa parameter controls exploration. Larger values reward uncertain regions more strongly.

29.4 Batch acquisition

Because GSIP evaluates candidates in parallel, batch acquisition functions such as qEI or noisy variants can propose several candidates jointly while accounting for redundancy.

29.5 Noisy observations

Even deterministic simulation can produce practical noise if approximate solvers, adaptive tolerances, stochastic domain models, or measured feedback are involved. Noisy Expected Improvement and robust GP modelling become relevant in those cases.

30. Constrained and mixed-variable Bayesian optimisation

Real problems include constraints and mixed variable types.

Constrained BO models objective and feasibility/constraint functions and seeks candidates that are both promising and likely feasible.

Mixed-variable BO handles continuous variables, integers, categories, and conditional choices. Naively encoding categories as integers creates false geometry. Better approaches use specialised kernels, one-hot/embedding structures, tree-structured search, or hybrid optimisers over the acquisition function.

Advanced mastery: high-dimensional BO

As dimension grows, standard GP BO degrades. Techniques include trust-region BO, additive models, random embeddings, sparse GPs, feature selection, and sensitivity-informed dimension reduction. GSIP's triage stage is strategically useful here: removing low-impact variables can make BO dramatically easier.

31. NSGA-II: non-dominated sorting genetic algorithm II

NSGA-II is an evolutionary algorithm for multi-objective optimisation. It maintains a population of candidate solutions, evaluates them, ranks them using Pareto dominance, preserves diversity using crowding distance, and creates new candidates using selection, crossover, and mutation.

The core cycle is:

1. initialise population;
2. simulate and score candidates;

3. perform non-dominated sorting;
4. calculate crowding distance;
5. select parents;
6. crossover and mutate;
7. evaluate offspring;
8. combine parents + offspring;
9. retain the best/diverse population;
10. repeat.

31.1 Pareto dominance

Solution A dominates B if A is no worse on all objectives and strictly better on at least one. Non-dominated solutions form the first Pareto front.

31.2 Non-dominated sorting

Candidates are grouped into fronts: front 1 is non-dominated; front 2 is dominated only by front-1 candidates; and so on. Better fronts receive higher selection priority.

31.3 Crowding distance

Crowding distance estimates how isolated a solution is in objective space. Keeping high-crowding candidates prevents the population from collapsing into one tiny part of the Pareto frontier.

31.4 Crossover and mutation

Crossover recombines features of parent solutions. Mutation introduces variation. Operators should be compatible with variable type and domain constraints. A good pack may provide repair functions for infeasible offspring.

31.5 Strengths

- robust multi-objective search;
- no gradients required;
- supports discontinuous/non-convex fronts;
- naturally produces diverse trade-off solutions.

31.6 Weaknesses

- can require many expensive evaluations;
- population size and operator tuning matter;
- no guarantee of global Pareto optimality;
- constraint handling can dominate performance;
- high-dimensional objective spaces make Pareto ranking less discriminative.

32. The Pareto front and decision making

A **Pareto front** contains solutions for which improving one objective requires worsening at least one other objective. It is often a better output than one scalar "best" solution.

For example:

- Design A: lowest cost;
- Design B: best cost/performance compromise;
- Design C: highest output;
- Design D: highest thermal margin.

None may dominate the others.

32.1 Hypervolume

Hypervolume measures the volume of objective space dominated by the Pareto set relative to a reference point. It is useful for comparing convergence and diversity of multi-objective optimisers. Improvements in hypervolume usually mean the front is moving toward better trade-offs and/or covering them more broadly.

32.2 Knee points

A knee point is a region where a small improvement in one objective requires a large sacrifice in another. These are often attractive compromise solutions, but identifying a knee does not replace user preference.

32.3 Human decision remains necessary

Optimisation can reveal trade-offs, but it cannot determine moral, strategic, or stakeholder preferences unless these are encoded as objectives or constraints. GSIP should present decision structure honestly rather than pretending the algorithm eliminates the human choice.

33. Why combine Bayesian optimisation with NSGA-II

The Build Brief preserves a cooperative hybrid rather than choosing only one algorithm.

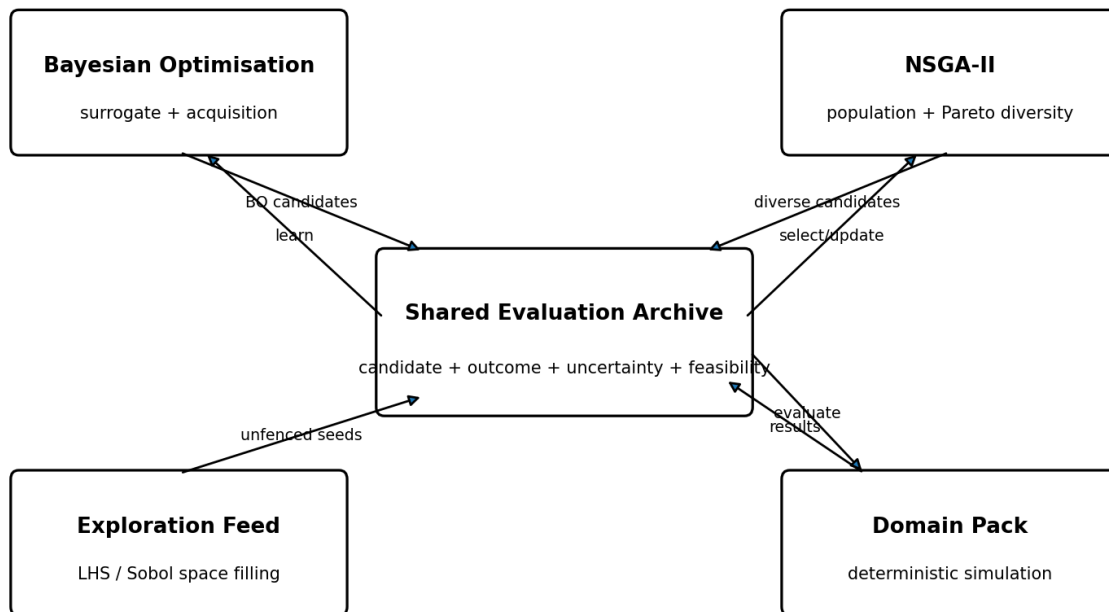
Bayesian optimisation contributes **sample efficiency**. It learns a surrogate from expensive evaluations and proposes candidates with high expected information/value.

NSGA-II contributes **population diversity and natural Pareto exploration**. It searches broadly and maintains trade-off coverage.

A shared evaluation archive lets each method benefit from every expensive simulation:

- BO injects acquisition-maximising candidates into the evaluation pool/NSGA-II population.
- NSGA-II contributes diverse evaluated candidates that improve the surrogate's coverage.
- Both consume the same deterministic simulation results instead of duplicating evaluations.

GSIP Hybrid Optimisation Loop



Every expensive simulation is shared: BO gains data efficiency; NSGA-II preserves multi-objective diversity.

Figure 2. Cooperative Bayesian optimisation and NSGA-II over one evaluation archive.

33.1 Why this combination is technically sensible

Using only NSGA-II can be evaluation-hungry when each simulation is expensive. Using only BO can struggle with complex mixed variables, many objectives, or highly irregular Pareto fronts. The hybrid tries to combine complementary strengths.

33.2 Advanced questions the implementation must answer

- How are evaluation budgets allocated between BO-proposed and evolutionary candidates?
- How are duplicate/near-duplicate candidates detected?
- What happens when the surrogate is poor early in the run?
- Which multi-objective acquisition function is used?
- How are constraints represented in both algorithms?
- How is uncertainty included in dominance?
- How does asynchronous completion update both methods?
- What stopping criterion defines convergence?

These are not merely library choices; they determine actual optimiser quality.

34. Multi-fidelity optimisation

The pack contract exposes model fidelity and cost, creating a natural path to **multi-fidelity optimisation**.

Instead of evaluating every candidate with the most expensive model, the optimiser can use a hierarchy:

- cheap toy/reduced-order model for broad exploration;
- medium-fidelity model for filtering;
- validated/high-fidelity model for finalists.

Advanced Bayesian methods can model correlations between fidelity levels and choose both **where** to evaluate and **at what fidelity**.

Potential benefit: orders-of-magnitude compute savings.

Potential risk: a cheap model may rank candidates incorrectly. Therefore fidelity correlation itself must be validated.

35. Ray and parallel/asynchronous evaluation

Ray is the proposed parallel execution framework. Simulation evaluations are often embarrassingly parallel because many candidates can be evaluated independently.

A single-node launch can fan out simulations across local CPU cores. Later, the same abstraction can scale to a Ray cluster/cloud environment without changing the pack interface.

35.1 Asynchronous completion

If one candidate takes 20 seconds and another takes 5 minutes, the optimiser should not necessarily wait for the slow candidate before learning from completed results. An asynchronous archive accepts out-of-order results and updates optimisation state as they arrive.

35.2 Reproducibility in distributed execution

Distributed evaluation requires stable candidate IDs, explicit seeds, idempotent tasks, and careful result reconciliation. The semantic result should not depend on which worker finishes first.

35.3 Fault tolerance

A robust runner should distinguish:

- domain infeasibility;

- simulation numerical failure;
- worker crash;
- timeout;
- infrastructure failure.

These statuses affect whether the optimiser retries, penalises a candidate, or aborts the run.

Part V - Software architecture and infrastructure

36. Pydantic and typed data contracts

Pydantic provides Python data validation and serialisable models. GSIP uses it for objects such as `ParameterValue`, `ProblemClassification`, `Playbook`, `TriageResult`, `ClarificationQuestion`, `DomainPackFidelity`, `OutcomeBundle`, and `ConfidenceReport`.

This is not cosmetic. A trust-sensitive pipeline should make illegal states difficult to represent.

Examples of useful validation include:

- confidence must be between 0 and 1;
- units/ranges must be valid;
- a `RETRIEVED` value should have a source identifier;
- a `COMPUTED` value should list parents;
- a `CALIBRATED` pack should provide calibration evidence metadata;
- gate verdicts must be from the defined enum.

Advanced implementations can use discriminated unions for different parameter types and custom validators for unit consistency.

37. FastAPI and the backend API

FastAPI is the proposed Python HTTP API layer. It integrates naturally with Pydantic and asynchronous workflows.

The architecture proposes endpoints such as:

- `POST /api/runs` - create a run;
- `POST /api/runs/{id}/clarify` - submit clarification/grounding information;
- `POST /api/runs/{id}/override` - perform an explicit logged gate override.

The separation of `/override` is important because bypassing a trust policy should be a deliberate, auditable action.

37.1 API design principles

A mature API should use idempotency keys for retried writes, explicit run-state models, structured error codes, versioned schemas, authentication/authorisation, and optimistic concurrency where humans may update the same run.

38. Workflow orchestration with Temporal

GSIP is not a simple synchronous request/response system. A run can pause at clarification or the provenance gate for minutes, hours, or days. **Temporal** is proposed as a durable workflow engine.

A conceptual run-state path is:

```
CREATED -> CLASSIFYING -> TRIAGING -> WAITING_FOR_GROUNDING -> GATED -> SIMULATING -> OPTIMISING -> REPORTING -> COMPLETE
```

A durable workflow engine provides persistence across worker restarts, retries, timeouts, and human waiting periods.

38.1 Why a persisted state machine is an acceptable temporary stand-in

The Build Brief allows a persisted asynchronous state machine if Temporal becomes a launch blocker. The architectural requirement is the durable pause/resume interface, not a specific vendor at all costs. If the simpler implementation preserves stable boundaries, it can be upgraded later.

38.2 Determinism in workflow code

Durable workflow systems often require workflow logic to be deterministic because histories are replayed. Side effects should occur in activities, while workflow code coordinates them. This maps well to the project's service layout: activities perform classification, retrieval, triage, grounding, gate decisions, scenario generation, and reporting.

39. PostgreSQL, SQLAlchemy, and Alembic

PostgreSQL is the proposed primary database. It can store run metadata, ledger records, pack registry data, playbook versions, clarification states, and evaluation archives.

SQLAlchemy provides Python database access/ORM abstractions.

Alembic provides schema migrations so the database evolves safely as models change.

39.1 Why relational storage fits the trust model

Provenance and audit data benefit from transactions, foreign keys, indexes, immutable/event tables, and queryable relationships. PostgreSQL also supports JSON columns for flexible metadata while preserving relational constraints for core entities.

39.2 Append-only ledger schema

A ledger event can include:

- event ID and monotonic sequence;
- run ID;
- event type;
- timestamp;
- actor/system component;
- payload;
- source hashes;
- previous-event hash for tamper evidence (advanced);
- schema version.

40. Embeddings, pgvector, and Milvus

An **embedding** maps an object such as text into a numerical vector whose geometry approximates semantic similarity.

Example:

text -> [0.17, -0.42, 0.91, ...]

Similarity search can retrieve playbooks whose wording differs from the user's request but whose meaning is related.

pgvector adds vector similarity search to PostgreSQL. It is the Build Brief's launch choice because Postgres is already required, reducing operational complexity.

Milvus is a specialised vector database that can be introduced later if scale or performance justifies it.

40.1 Stable retrieval interface

The core should depend on an interface such as `retrieval_store.search()` rather than on Milvus-specific calls. This allows pgvector -> Milvus replacement without redesigning callers.

40.2 Retrieval risks

Embedding similarity is not proof of applicability. A semantically similar playbook may still be invalid because of regime, jurisdiction, unit system, model assumptions, or domain nuance. Retrieval results should therefore pass structured applicability checks.

41. React, TypeScript, and the honest-output interface

The frontend is not merely a chat window. The architecture needs an interface that makes trust visible.

React provides component-based interactive UI development.

TypeScript adds static typing to JavaScript, making API contracts safer and reducing state-management errors.

A strong GSIP run screen should expose:

- classified frames and confidence;
- selected playbook/version;
- critical parameters;
- provenance status for each important value;
- questions/actions required to unblock the run;
- model fidelity and validation status;
- simulation/optimisation progress;
- Pareto trade-offs;
- uncertainty bands;
- overrides and caveats;
- links to the audit trail.

The UI is part of the trust architecture. If warnings exist only in backend metadata and the user sees one polished number, the system has failed its product goal.

42. Docker, Docker Compose, and reproducible packaging

Docker packages the application and dependencies into reproducible containers. This helps standardise Python versions, system libraries, database clients, optimiser dependencies, and runtime environments.

Docker Compose can start the local development stack - API, database, workers, frontend, and optional workflow services - with a single command.

The project explicitly values reproducible single-command runs.

42.1 Container images as provenance

For high-assurance runs, the exact image digest can be stored in the run ledger. This records not merely "version 1.2" but the precise environment used for the simulation.

43. YAML and configuration-as-code

YAML run specifications make experiments reviewable and reproducible.

A run specification can include:

```
pack: generator_v2
seed: 42
objectives:
  - maximize_power
  - minimize_cost
constraints:
  max_temperature: 90
budget:
  evaluations: 200
```

Configuration-as-code supports version control, code review, experiment comparison, and deterministic replay. A strong implementation should validate YAML through Pydantic rather than trusting arbitrary keys.

44. LLM adapter and provider independence

The Build Brief proposes a single LLM adapter so the system does not become coupled to one provider. Higher layers should call a stable interface for structured classification, drafting, scenario proposal, and explanation.

Benefits include:

- provider/model swaps without redesign;
- centralised logging and safety settings;
- common retry/timeout policy;
- easier A/B testing;
- support for local models later;
- consistent structured-output validation.

Advanced implementation should record model identifier, prompt template version, generation parameters, and response hash in the run ledger.

45. Sandboxed execution of user-authored packs

User-authored packs execute arbitrary code and therefore form a serious security boundary.

The launch choice is a subprocess sandbox with resource and time limits. A stronger future isolation layer can use per-pack containers or technologies such as gVisor.

Threats include:

- filesystem access;
- secret/environment-variable theft;
- network exfiltration;
- denial of service through CPU/memory exhaustion;
- fork bombs;
- kernel exploitation;
- dependency supply-chain attacks.

45.1 Security controls

A mature hosted system should consider:

- no network by default;
- read-only filesystem except controlled scratch space;
- CPU/memory/pid limits;
- wall-clock timeout;
- seccomp/capability restrictions;
- separate UID/namespace;

- signed pack artefacts;
- dependency allowlists or locked environments;
- output size limits;
- malware/static scanning;
- secrets never mounted into pack containers;
- strong audit logging.

Sandboxing is not optional if the product becomes an open pack ecosystem.

46. Pack SDK, scaffolding, validation, and registry

An **SDK** is a software development kit that makes extension practical for developers who do not know the entire GSIP codebase.

The Build Brief treats the pack-authoring experience as a headline feature. A scaffolding command such as `gsip new-pack` can generate templates, tests, fidelity metadata, and a playbook skeleton.

A validator should check at minimum:

- contract methods exist;
- schemas are valid;
- deterministic replay passes;
- `simulate()` returns a valid `OutcomeBundle`;
- fidelity, validation status, applicability bounds, limitations, and references exist;
- cost model is declared;
- tests pass;
- package has no prohibited capabilities in hosted mode.

46.1 LLM-assisted authoring

The model can draft schemas, playbooks, tests, and documentation from a user's domain description. Human experts must review and ratify knowledge that affects framing or scientific validity. This preserves the trust architecture while reducing authoring friction.

46.2 Registry

A pack registry should store identity, version, author, dependencies, compatibility, fidelity, validation evidence, security status, and supported playbooks. Discovery should not require modifying the core code.

47. The run ledger and event sourcing

The run ledger is an append-only audit of what happened. A useful conceptual model is **event sourcing**, where state can be reconstructed from an ordered sequence of events.

Events might include:

- `run_created`;
- `classification_completed`;
- `playbook_selected`;
- `triage_completed`;
- `parameter_grounding`;
- `gate_blocked`;
- `clarification_answered`;
- `override_applied`;
- `scenario_proposed`;
- `simulation_completed`;
- `optimiser_updated`;

- report_generated.

Benefits include debugging, forensic review, replay, provenance explanation, and regulatory evidence.

Advanced tamper-evidence can chain event hashes, but cryptographic integrity should be designed carefully rather than assumed from append-only database permissions alone.

Part VI - Testing and engineering discipline

48. Unit testing the provenance model and gate

The gate is small enough that its full matrix should be exhaustively tested. Tests should cover every combination of criticality and provenance, including computed values with weak parents and explicit override paths.

Property tests can assert invariants such as:

- a CRITICAL + LLM_RECALL value can never silently produce RUN;
- a computed value can never become stronger than its weakest parent;
- overrides always appear in the confidence report and ledger;
- identical run specs/seeds produce identical scenario identities.

These are excellent candidates for property-based testing because the rules are structural.

49. Classifier and calibration testing

Classifier evaluation requires more than accuracy.

Test suites should include:

- per-label precision/recall/F1;
- multi-label metrics;
- calibration curves and ECE/Brier score;
- abstention coverage vs selective risk;
- adversarially ambiguous queries;
- out-of-domain queries;
- frame-disagreement cases;
- regression sets for known failures.

The most important production metric may be the cost of **false confidence** rather than average accuracy.

50. Sensitivity and triage testing

Morris screening should be validated on analytic functions with known influential variables. Tests should verify that critical variables remain critical across seeds/ranges and that low-impact variables are not promoted due to implementation artefacts.

For domain packs, compare Morris ranking with deeper Sobol or local derivative analysis on representative cases where compute allows.

51. Optimiser validation with ZDT and DTLZ benchmarks

The Build Brief says ZDT/DTLZ should remain an internal optimiser-validation harness.

ZDT and **DTLZ** are families of synthetic multi-objective optimisation problems with known or well-characterised Pareto fronts. They are useful because the optimiser can be tested independently of any domain pack.

Metrics can include:

- hypervolume;
- generational distance/inverted generational distance;
- Pareto-front coverage/diversity;
- number of evaluations to reach a quality threshold;
- reproducibility across seeds;
- behaviour under parallel/asynchronous execution.

The benchmark harness is not the product. It is evidence that the generic optimiser works without domain-specific assumptions.

52. Pack validation and numerical testing

Every pack should have:

- schema tests;
- feasibility tests;
- deterministic replay tests;
- analytic/simple reference cases;
- regression fixtures;
- boundary tests;
- invalid-input tests;
- numerical failure tests;
- fidelity/applicability tests;
- performance-cost tests.

Validated or calibrated packs need additional evidence such as comparison with independent ground truth, error distributions, and documented acceptance criteria.

53. Integration and end-to-end testing

An end-to-end test should exercise:

question -> classification -> playbook -> triage -> grounding -> gate -> scenario -> simulation -> optimiser -> report

Important test scenarios include:

- critical ungrounded value -> BLOCK;
- user clarification -> resume;
- explicit override -> run but report illustrative;
- unknown problem -> ABSTAIN;
- toy pack -> report cannot say predictive;
- worker failure -> retry without duplicated ledger semantics;
- out-of-order Ray results -> same archive semantics;
- same spec/seed -> reproducible results.

Part VII - How the project can be used

54. Engineering design optimisation

GSIP is naturally suited to engineering domains where a deterministic or validated simulator already exists. Examples include electromechanical systems, thermal systems, structures, energy systems, control settings, and process engineering.

The user can express goals in natural language, while the pack formalises variables and constraints. The optimiser can return a Pareto front rather than one arbitrary design.

The strongest use is not "invent physics." It is **search a trusted model more intelligently and explain the trade-offs while enforcing evidence quality.**

55. Generator optimisation example

The architecture's generator example uses relations such as:

$$P = V * I$$

and a simplified Faraday relationship:

$$V = N * d\Phi/dt$$

where P is electrical power, V voltage, I current, N number of turns, and Phi magnetic flux.

Increasing output is not one-dimensional. More current increases resistive losses approximately as:

$$P_{loss} = I^2 * R$$

More turns can increase induced voltage but also resistance, material use, size, and thermal behaviour. Higher RPM may raise power while increasing losses, mechanical stress, and thermal constraints.

This is exactly why multi-objective search is valuable.

A reduced-order pack can provide directional optimisation, while a validated finite-element/digital-twin pack could support stronger claims if grounded inputs and applicability are satisfied.

56. Environmental and policy scenario analysis

A dispersion/pollution pack can compare intervention scenarios, but the architecture deliberately prevents the system from inventing critical emission-source coordinates or intensities.

The strongest use is scenario ranking when:

- a real emission inventory is supplied;
- weather/flow fields are grounded;
- model fidelity is honestly declared;
- outputs are interpreted within model limits.

For policy problems, multiple frames may be required: physical effect, economic cost, public health, and feasibility. GSIP can surface that the "best physical intervention" and "best feasible intervention" are not the same question.

57. Supply-chain and operations optimisation

A supply-chain pack can model:

- supplier capacities and costs;
- demand;
- lead times;

- inventory;
- transport modes;
- warehouse capacities;
- service targets;
- disruption scenarios.

Objectives may include cost, delivery time, stockout risk, carbon impact, and resilience. Provenance is especially valuable because operational decisions often depend on data from many internal systems whose freshness and source matter.

58. Manufacturing and energy systems

Manufacturing packs can optimise throughput, energy, maintenance windows, quality, and material usage.

Energy packs can optimise dispatch, storage, charging, load shifting, or component sizing.

These domains benefit from simulation-in-the-loop because constraints are complex and evaluations may come from established engineering software rather than from formulas generated by the AI.

59. Scientific parameter search

Researchers often have expensive simulators such as CFD, finite-element, chemical-process, climate-subsystem, or biological models. GSIP can wrap these models with sample-efficient search and provenance-controlled input handling.

The major value is reducing the number of expensive simulations while preserving a reproducible archive of what was tried and why.

60. Enterprise private decision models

An organisation can package private cost models, operational constraints, digital twins, and internal datasets as packs. Employees then interact through natural language while final numerical decisions remain grounded in company-specific code and data.

This can be more defensible than asking a general-purpose LLM to reason from incomplete company context.

61. Finance and other high-stakes domains

The architecture can technically host deterministic financial or risk models, but the presence of a pack does not remove regulatory, model-risk, suitability, or data-quality obligations. High-stakes domains require stricter validation, source governance, human review, and compliance controls.

The correct lesson is not "GSIP makes any decision safe." The architecture makes evidence and model limitations more explicit, which can improve governance when combined with domain-specific controls.

Part VIII - Drawbacks, limitations, and failure modes

62. Open-set classification is genuinely hard

The architecture itself identifies reliable abstention as the riskiest single component. A classifier can be confident on unfamiliar input, and simple probability thresholds do not solve this permanently.

Failure modes:

- false confident classification -> wrong playbook and wrong variables;
- unnecessary abstention -> frustrating coverage loss;
- ambiguous multi-frame request -> silently optimising the wrong objective.

Mitigations include ensembles, retrieval agreement, OOD detectors, selective prediction metrics, human disambiguation, and continuous failure-set curation.

63. Playbook curation is expensive and can become stale

Human-ratified playbooks are a strength and a bottleneck. Coverage grows only as quickly as trusted knowledge is created and maintained.

Drawbacks:

- expert review cost;
- version management;
- standards/research change;
- domain disagreements;
- unclear ownership.

Mitigations: structured templates, review workflows, expiry/review dates, LLM drafting with expert approval, usage analytics, and deprecation mechanisms.

64. Retrieval does not guarantee correct data

A wrong-but-cited dataset can still produce a wrong result. Provenance improves traceability but cannot guarantee source quality.

Mitigations:

- source-quality metadata;
- cross-source checks;
- freshness rules;
- validation against known ranges;
- user confirmation for conflicting sources;
- explicit uncertainty widening.

65. Sensitivity screening costs compute and can mislead

Morris is cheaper than full Sobol analysis, but expensive simulators can make even screening costly. Results also depend on tested ranges and can miss context-specific interactions.

Mitigations:

- playbook priors before empirical screening;
- cheaper fidelity models;
- surrogate-based sensitivity;
- adaptive screening;

- caching sensitivity results for similar states;
- escalation to deeper analysis only when needed.

66. Fidelity tiers are coarse and partly self-reported

REDUCED_ORDER can describe models with very different accuracy. A four-level tier is a communication tool, not an error bound.

Mitigations:

- attach quantitative validation metrics;
- independent review;
- model cards;
- pack certification;
- automatic applicability checks;
- separate evidence grade from fidelity category.

67. Ground-truth calibration is often unavailable

Counterfactual, one-shot, and long-horizon decisions may never produce a clean ground-truth comparison. Such packs may remain unvalidated indefinitely.

The correct response is honest uncertainty, not fabricated calibration.

68. Optimisation algorithms do not guarantee the true global optimum

BO and NSGA-II are powerful search methods, not universal proofs. Performance depends on evaluation budget, dimension, constraints, surrogate quality, initialisation, and landscape structure.

Mitigations:

- benchmark validation;
- multiple seeds/restarts;
- hybrid search;
- convergence diagnostics;
- wider exploration budgets;
- local refinement;
- domain-specific optimisation backends behind the stable interface.

69. The surrogate model can be confidently wrong

BO relies on a statistical approximation of the expensive simulator. Sparse data, poor kernels, discontinuities, and mixed variables can create misleading uncertainty.

Mitigations include surrogate diagnostics, ensembles, trust regions, minimum exploration, model comparison, and fallback evolutionary/random exploration.

70. Provenance introduces UX and data-engineering overhead

Tracking every important value, source, uncertainty, and lineage is more work than accepting a chatbot answer. Users may experience extra questions and integration effort.

This is a deliberate trade-off: trust is not free.

Good triage and pack tooling are essential so the system spends user attention only on variables that materially affect the answer.

71. User-authored code creates a major security surface

A plugin ecosystem can become an arbitrary-code execution platform. Strong isolation, dependency management, and permission policy are required before exposing it to untrusted authors.

This can become one of the largest operational costs of the product.

72. Infrastructure complexity

The full stack contains API, database, vector retrieval, workflow orchestration, parallel compute, optimisation libraries, frontend, containerisation, and sandboxing. Building every production-grade component at once would create a high execution risk.

This is why the Build Brief uses launch substitutions such as pgvector before Milvus and single-node Ray before a cluster. Stable interfaces preserve ambition while controlling build complexity.

73. Audit logs and evaluation archives can grow rapidly

Parallel optimisation can create thousands or millions of evaluation records, simulation artefacts, and provenance events.

The system will eventually need retention policies, summarised views, cold storage, artifact compression, and indexed query paths.

74. Causal claims are not automatically justified

Optimisation over a simulator identifies the best action *according to that model*. It does not automatically establish real-world causality. If a pack is built from correlational models, interventions may fail under distribution shift.

Causal inference must be an explicit property of the domain model/playbook, not an assumption added by the GSIP core.

75. Multi-objective optimisation does not eliminate human values

A Pareto front can show trade-offs, but choosing between cost, safety, performance, fairness, environmental impact, or policy feasibility remains a stakeholder decision unless preferences are formally encoded.

A trustworthy system should expose this remaining human judgement instead of pretending the optimiser "decided objectively."

76. Privacy, compliance, and confidential data

Grounding may require sensitive organisational data. The system therefore needs access control, encryption, tenant isolation, retention policy, data minimisation, and potentially on-prem/local deployment options.

A provenance ledger itself may contain sensitive information and must be protected accordingly.

Part IX - Advanced extensions for mastering the subject

77. Robust optimisation

Standard optimisation often assumes parameter estimates are fixed. **Robust optimisation** seeks decisions that perform acceptably across uncertainty sets.

Instead of maximising $f(x, \theta_{\hat{}})$ at one estimated parameter vector, optimise worst-case or risk-adjusted performance across plausible θ .

This is attractive for GSIP because provenance already identifies uncertain inputs. Critical values with broad uncertainty can drive robust candidate selection rather than merely wider report bands.

78. Chance-constrained optimisation

A chance constraint requires a constraint to hold with high probability, for example:

$P(\text{temperature} \leq 90 \text{ C}) \geq 0.99$

This is more realistic than enforcing a deterministic temperature point estimate when inputs are uncertain.

Implementation requires probabilistic outcome models or repeated/surrogate uncertainty evaluation.

79. Distributionally robust optimisation

When the exact uncertainty distribution is itself uncertain, **distributionally robust optimisation (DRO)** optimises against a family of plausible distributions. This can protect against model misspecification and dataset shift.

It is an advanced natural extension of GSIP's "trustworthy about trustworthiness" philosophy.

80. Active learning and value of information

Triage currently asks which parameters matter. A deeper question is:

> Which measurement or clarification would most improve the decision?

Value of information (VOI) methods quantify the expected benefit of reducing uncertainty in a variable. This can optimise the grounding process itself.

Example: if measuring temperature coefficient costs \$1,000 but is expected to change a \$1M decision substantially, it may be worth obtaining. If another measurement will not affect the decision, skip it.

This turns clarification from sensitivity ranking into economically optimal evidence acquisition.

81. Bayesian calibration

Instead of fitting a single model parameter value, **Bayesian calibration** represents posterior distributions over uncertain parameters after comparing model predictions with observations.

This produces richer uncertainty estimates and can distinguish parameter uncertainty from residual/model discrepancy. It is computationally heavier but valuable for high-fidelity packs.

82. Conformal prediction

Conformal prediction provides distribution-free coverage guarantees under exchangeability assumptions. For suitable supervised surrogate or predictive components, conformal intervals can complement model-based uncertainty.

It should not be used as a magical universal guarantee: coverage can fail under distribution shift, and the guarantees concern the statistical prediction layer, not the truth of a simulator's structural assumptions.

83. Multi-objective Bayesian optimisation

Instead of using BO only to inject promising points into NSGA-II, the system can adopt full multi-objective BO using acquisition functions such as expected hypervolume improvement.

The acquisition asks which candidate is expected to increase the dominated hypervolume of the Pareto set the most.

This can be highly sample-efficient for expensive multi-objective simulators, although computational complexity grows with objectives, batch size, and constraints.

84. Trust-region Bayesian optimisation

For high-dimensional problems, global GP models often struggle. **Trust-region BO** restricts search to a local region around promising points and adapts region size according to success/failure.

Triage can work synergistically with this: remove low-impact dimensions first, then apply trust-region BO in the influential subspace.

85. Surrogate-assisted evolutionary algorithms

Another advanced hybrid is to use surrogates to predict which evolutionary candidates are worth expensive simulation. The evolutionary algorithm generates diversity; a surrogate pre-screens candidates; only promising/uncertain candidates reach the real simulator.

This is closely aligned with the project's cooperative BO + NSGA-II philosophy.

86. Provenance DAGs and source-quality models

The current provenance taxonomy is categorical. A mature implementation can build a provenance DAG where every derived value links to parent values and source artifacts.

A separate **source-quality model** can score dimensions such as:

- authority;
- freshness;
- measurement calibration;
- sampling quality;
- scope/applicability;
- conflict with other sources.

The gate can then use provenance class plus quality, rather than treating every retrieved source equally.

87. Formal verification and executable policy invariants

The most critical rules can be expressed as invariants and verified with property-based tests or formal methods.

Example invariant:

```
criticality == CRITICAL and effective_provenance == LLM_RECALL -> verdict != RUN
```

More advanced systems can use typed state machines so a run cannot enter SIMULATING unless the gate has produced an allowed state.

88. Pack signing, supply-chain security, and reproducible builds

A mature pack ecosystem should know exactly what code ran. Advanced controls include signed pack releases, SBOMs (software bills of materials), pinned dependency hashes, reproducible container builds, vulnerability scanning, and revocation of compromised pack versions.

This is the software-security analogue of data provenance.

89. Observability

Production observability should include metrics, logs, and traces for:

- classification latency and abstention rates;
- grounding time;
- gate block reasons;
- simulation latency/failure rates;
- Ray worker utilisation;
- optimiser progress/hypervolume;
- database/ledger growth;
- pack-specific failure rates;
- report caveat rates.

This allows the team to measure whether the architecture works in practice, not merely in design.

90. Cost-aware orchestration

Because simulations, retrieval, and model calls all cost money/time, GSIP can eventually optimise the workflow budget itself. A run controller can decide whether the expected value of another simulation, higher-fidelity evaluation, or data retrieval justifies its cost.

This turns the system into a decision engine not only over user actions but also over its own computation.

Part X - Project roadmap, best uses, and definition of success

91. Build phases in the handoff

The Build Brief expands the architecture into a staged launch plan. In practical terms:

Phase 0 - Ground truth and scaffolding. Inventory the existing repository and PDFs, perform a gap analysis, establish the monorepo, Docker, Postgres, FastAPI skeleton, CI, and confirm the existing optimiser on ZDT/DTLZ benchmarks.

Phase 1 - Provenance spine and gate. Implement ParameterValue, provenance taxonomy, weakest-parent COMPUTED inheritance, gate matrix, run-ledger logging, and unit tests. Everything downstream depends on this.

Phase 2 - Pack contract and SDK. Implement/extend DomainPackBase, fidelity descriptor, discovery/registry, validator, sandboxed execution, and reference fixture packs proving genericity.

Phase 3 - Optimiser integration. Preserve the v1 hybrid BO + NSGA-II engine behind the generic pack interface, shared archive, split-budget scenario feed, Ray parallel evaluation, and uncertainty-aware Pareto representation.

Phase 4 - Triage, grounding, and clarification. Add Morris screening, criticality ranking, triage-driven questions, provenance resolution, and cheap uncertainty propagation.

Phase 5 - Classifier, playbooks, and registry. Add multi-label calibrated classification, abstention/disambiguation, playbook schema/versioning, and routing across registered packs.

Phase 6 - Orchestration and human-in-the-loop. Implement durable pause/resume at gate and clarification plus run/clarify/override API flows.

Phase 7 - Confidence report and honest-output UI. Propagate fidelity, provenance, uncertainty, overrides, and Pareto trade-offs into the report and React interface.

Phase 8 - Pack-authoring experience. Deliver scaffolding, authoring UI, LLM-assisted pack/playbook drafting with human ratification, registry sharing, and hardened sandboxing.

Phase 9 - Non-blocking upgrades. Replace launch simplifications only when needed: pgvector -> Milvus, prompted LLM -> fine-tuned SLM, stronger calibration, additional optimiser backends, cluster compute, stronger sandbox isolation.

The build philosophy is: **reduce the complexity of building the first version, never reduce the long-term design ambition.**

92. Definition of a credible launch

A credible launch is not merely "the API runs." A user should be able to:

1. register or author a pack in their own domain;
2. pose a natural-language problem;
3. see how the system classified/framed it;
4. see which parameters are decision-critical;
5. experience a visible BLOCK/FLAG when a critical input is ungrounded;
6. ground or explicitly override the input;
7. run deterministic simulations from the pack;
8. obtain an uncertainty-aware Pareto/trade-off result;
9. see fidelity, validation, assumptions, and provenance in the final report;
10. reproduce/audit the run.

That end-to-end behaviour is what demonstrates the platform thesis.

93. Where GSIP is strongest

GSIP is strongest when all of the following are true:

- the decision is important enough to justify explicit evidence;
- a deterministic/validated domain model exists or can be written;
- simulation calls are expensive enough that intelligent search matters;
- multiple objectives or constraints create non-trivial trade-offs;
- important inputs come from identifiable sources;
- auditability and reproducibility are valuable;
- users need a natural-language interface to complex modelling infrastructure.

94. Where GSIP is a poor fit

GSIP may be unnecessary or inappropriate when:

- the task is simple enough for direct calculation;
- no meaningful computational model exists;
- the decision is purely subjective/creative;
- obtaining grounded inputs is impossible and the user wants a definitive prediction;
- simulation is so cheap and search space so small that brute force is simpler;
- the cost of pack/playbook curation exceeds the value of the decision.

Knowing when *not* to use the architecture is part of mastery.

95. The product moat and long-term value

Bayesian optimisation, NSGA-II, language models, databases, and workflow engines are individually available technologies. The differentiated value is the integration of:

- a domain-pack ecosystem;
- curated/versioned playbooks;
- parameter sensitivity triage;
- data lineage and provenance gating;
- model fidelity and validation discipline;
- sample-efficient multi-objective optimisation;
- parallel execution;
- uncertainty-aware reporting;
- durable human-in-the-loop workflow;
- an auditable run ledger;
- a first-class authoring experience.

A mature registry of trusted packs and playbooks can become particularly valuable because it represents reusable executable domain knowledge, not merely generic model capability.

Part XI - Mastery reference

96. Core equations and concepts quick reference

Deterministic simulator

$$y = f(x, a)$$

State x and action a produce outcome y through code.

First-order uncertainty propagation

$$\text{Var}(Y) \text{ approximately } \sum ((dY/dX_i)^2 \text{Var}(X_i))$$

under local linearity and independence assumptions.

Morris elementary effect

$$EE_i = [f(x + \Delta e_i) - f(x)] / \Delta$$

Sobol first-order sensitivity

$$S_i = \text{Var}_{X_i}(E[Y | X_i]) / \text{Var}(Y)$$

Gaussian-process posterior concept

$$f(x) \sim \text{Normal}(\mu(x), \sigma^2(x))$$

Upper confidence bound

$$UCB(x) = \mu(x) + \kappa \sigma(x) \text{ for maximisation.}$$

Pareto dominance

A dominates B if A is no worse in every objective and strictly better in at least one.

Electrical power

$$P = V I$$

Resistive loss

$$P_{\text{loss}} = I^2 R$$

Simplified induced voltage relation

V proportional to $N d\Phi/dt$

97. Glossary

Term	Meaning in this project
Abstention	The classifier explicitly declines to select a validated frame.
Acquisition function	BO rule for selecting the next valuable expensive evaluation.
Aleatoric uncertainty	Inherent variability/noise in observations or system behaviour.
API	Programmatic interface between frontend, workflow, and services.
Applicability bounds	Conditions/ranges in which a model/playbook is valid.
Auditability	Ability to reconstruct why and how a run produced its result.
Bayesian optimisation	Sample-efficient optimisation using a surrogate and acquisition rule.
BoTorch	Python/PyTorch ecosystem used for Bayesian optimisation.

GSIP Complete Technical Master Guide

Term	Meaning in this project
Brier score	Proper scoring rule measuring probabilistic prediction error.
Calibration	Alignment between predicted probabilities/model outputs and observed reality.
Clarification	User-facing process for grounding missing important parameters.
Computed provenance	Value deterministically derived from parents; inherits weakest parent.
Confidence report	Structured summary of fidelity, provenance, uncertainty, overrides, verdict.
Constraint	Rule restricting feasible candidate solutions.
Cost model	Pack method estimating compute cost at each fidelity.
Crowding distance	NSGA-II diversity measure in objective space.
Criticality	How strongly a parameter affects the decision outcome.
Determinism	Same specified input produces the same output.
Domain pack	Plugin containing domain-specific executable simulation/scoring logic.
DTLZ	Synthetic multi-objective benchmark family used to validate optimisers.
ECE	Expected Calibration Error.
Embedding	Numerical vector representation used for similarity retrieval.
Epistemic uncertainty	Uncertainty due to incomplete knowledge/model/data.
Exploit	Candidate generation focused on promising/plausible regions.
Explore	Candidate generation focused on broad discovery outside prior expectations.
FastAPI	Python web framework for the backend API.
Feasibility	Whether a state/action satisfies physical/operational constraints.
Fidelity	How faithful the model is to the real system for intended use.
Gaussian process	Probabilistic function model commonly used as a BO surrogate.
Gate	Policy combining criticality and provenance into RUN/FLAG/BLOCK.
Grounding	Resolving a value to an attributable source or explicit assumption.
Hypervolume	Multi-objective quality metric for Pareto coverage and convergence.
Isotonic calibration	Non-parametric monotonic probability calibration method.
Latin Hypercube Sampling	Stratified multidimensional space-filling design.
Ledger	Append-only audit history of a run.
LLM	Large Language Model used for framing, drafting, proposing, explaining.
LLM_RECALL	Value generated from model memory without attributable source.
Milvus	Specialised vector database considered as a later retrieval backend.
Morris method	Efficient elementary-effects global sensitivity screening.
Multi-fidelity	Use of several model fidelities/cost levels during search.

GSIP Complete Technical Master Guide

Term	Meaning in this project
Multi-label classification	Classification where multiple problem frames may be active.
Mutation	Evolutionary search operator adding variation.
NSGA-II	Multi-objective evolutionary algorithm using non-dominated sorting/crowding.
OOD	Out of distribution; input unlike validated/training examples.
OutcomeBundle	Simulation metrics plus uncertainty and model fidelity.
Override	Explicit user-authorised bypass of a blocked gate, fully logged/caveated.
Pareto front	Set of non-dominated trade-off solutions.
ParameterValue	Value object carrying origin, uncertainty, lineage, and notes.
pgvector	PostgreSQL extension for vector similarity search.
Playbook	Curated/versioned method for a class of problems.
PostgreSQL	Primary relational database for runs, ledger, registry, state.
Provenance	Origin and derivation history of a value.
Pydantic	Typed validation models for service/data contracts.
Ray	Python parallel/distributed execution framework for simulation fan-out.
Reduced-order model	Simplified model retaining dominant behaviour at lower compute cost.
Reproducibility	Ability to recreate a run from its specification/environment.
Retrieval	Obtaining attributable information from a named source/store/tool.
Robust optimisation	Optimisation seeking performance across uncertainty, not one estimate.
Sandbox	Isolation boundary for untrusted user-authored pack code.
Scenario	Candidate decision/configuration proposed for simulation.
Selective prediction	Model answers only when confidence/acceptance policy is satisfied.
Sensitivity	Degree to which an input changes an output.
SLM	Small Language Model, potentially specialised for classification.
Sobol sequence	Low-discrepancy quasi-random sequence for space-filling exploration.
Sobol sensitivity index	Variance-based measure of parameter influence.
SQLAlchemy	Python database toolkit/ORM.
Surrogate model	Cheap statistical approximation of an expensive simulator/objective.
Temperature scaling	Single-parameter calibration applied to classification logits.
Temporal	Durable workflow orchestration engine for pause/resume/retries.
Triage	Ranking required parameters by decision influence.
Validation	Evidence that a model is sufficiently representative for intended use.

Term	Meaning in this project
Vector database	Store/index optimised for similarity search in embedding space.
YAML	Human-readable configuration format for reproducible run specifications.
ZDT	Synthetic multi-objective benchmark family used to test optimisers.

98. Questions you should be able to answer after mastering GSIP

1. Why is deterministic simulation necessary but insufficient for trustworthy decisions?
2. What is the difference between provenance and source quality?
3. Why must COMPUTED inherit the weakest parent provenance?
4. Why is multi-label classification better than single-label classification here?
5. What does probability calibration change operationally?
6. Why is abstention a feature rather than a failure?
7. What is the difference between a playbook and a domain pack?
8. Why does triage precede grounding?
9. What does Morris screening estimate and why is it cheaper than Sobol decomposition?
10. Why are Sobol sequences and Sobol sensitivity analysis different concepts?
11. Why does CRITICAL + LLM_RECALL lead to BLOCK?
12. Why is a declared default different from model recall?
13. Why does a scenario proposed by the LLM still need provenance?
14. What is the role of a fidelity descriptor?
15. What is the difference between verification, validation, and calibration?
16. How does Bayesian optimisation reduce expensive simulator calls?
17. What role does a Gaussian-process uncertainty estimate play in acquisition?
18. What is Pareto dominance?
19. Why does NSGA-II use crowding distance?
20. Why can BO and NSGA-II be complementary?
21. What is the purpose of the shared evaluation archive?
22. Why does Ray fit the simulation workload?
23. Why does GSIP need durable workflow orchestration?
24. Why is the run ledger append-only?
25. Why is pgvector a reasonable launch substitution for Milvus?
26. Why must pack execution be sandboxed?
27. Why should the UI expose provenance and fidelity directly?
28. Which parts of the system are generic and which are domain-specific?
29. What does "trustworthy about its own trustworthiness" mean?
30. Under what conditions should GSIP refuse to claim a prediction?

99. Suggested mastery progression

Level 1 - Project understanding. Be able to explain the ten stages, provenance gate, domain-pack split, and fidelity concept without notes.

Level 2 - Algorithm literacy. Implement small examples of Morris screening, LHS/Sobol sampling, Gaussian-process BO, and NSGA-II on synthetic functions.

Level 3 - Systems implementation. Build typed Pydantic contracts, a FastAPI run service, Postgres ledger, and a durable pause/resume state machine.

Level 4 - Domain-pack engineering. Create a pack with schemas, deterministic simulator, feasibility checks, cost model, fidelity metadata, tests, and validation evidence.

Level 5 - Trust engineering. Implement provenance inheritance, gate policy, source-quality metadata, uncertainty propagation, and honest UI/reporting.

Level 6 - Advanced optimisation. Study constrained BO, multi-objective BO, robust optimisation, multi-fidelity methods, surrogate-assisted evolutionary algorithms, and value of information.

Level 7 - Production governance. Master sandboxing, software supply-chain security, model validation, playbook governance, observability, privacy, and audit retention.

100. Foundational references for further study

The following are useful starting points for deeper mastery of the underlying techniques:

- Morris, M. D. (1991). *Factorial Sampling Plans for Preliminary Computational Experiments*. Technometrics, 33(2), 161-174. - Foundation for the Morris elementary-effects sensitivity method.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). *A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II*. IEEE Transactions on Evolutionary Computation, 6(2), 182-197.
- Jones, D. R., Schonlau, M., & Welch, W. J. (1998). *Efficient Global Optimization of Expensive Black-Box Functions*. Journal of Global Optimization, 13, 455-492. - Classic expected-improvement Bayesian optimisation.
- Rasmussen, C. E., & Williams, C. K. I. (2006). *Gaussian Processes for Machine Learning*. MIT Press.
- McKay, M. D., Beckman, R. J., & Conover, W. J. (1979). *A Comparison of Three Methods for Selecting Values of Input Variables in the Analysis of Output from a Computer Code*. Technometrics, 21(2), 239-245. - Latin Hypercube Sampling.
- Saltelli, A., Ratto, M., Andres, T., et al. (2008). *Global Sensitivity Analysis: The Primer*. Wiley. - Variance-based/Sobol sensitivity analysis.
- Zitzler, E., Deb, K., & Thiele, L. (2000). *Comparison of Multiobjective Evolutionary Algorithms: Empirical Results*. Evolutionary Computation, 8(2), 173-195. - Includes the ZDT benchmark family context.
- Deb, K., Thiele, L., Laumanns, M., & Zitzler, E. (2002). *Scalable Multi-Objective Optimization Test Problems*. Congress on Evolutionary Computation. - DTLZ benchmark family.

For software implementation, study the official documentation of Pydantic, FastAPI, PostgreSQL, SQLAlchemy, Alembic, Ray, Temporal, React, TypeScript, Docker, BoTorch, pymoo, pgvector, and the selected vector-store/sandbox technologies. The project should avoid coupling itself to specific current versions unless the repository pins them.

101. Final synthesis

GSIP's deepest idea is not any single algorithm. It is the decision to build a hard architectural boundary between **language intelligence** and **numerical authority**.

The language model is excellent at interpreting what a human means, retrieving reusable structure, asking useful questions, proposing plausible possibilities, and explaining complex results. It is not a measuring instrument. The deterministic domain pack is excellent at computing what a specified model says. It does not know whether its inputs are real. The optimiser is excellent at searching a formal design space. It does not know whether the model deserves to be trusted. The provenance system knows where values came from, but not which variables matter. Sensitivity analysis knows which variables matter, but not whether the sources are trustworthy. Fidelity tells the user how much authority the model deserves, but it does not make the inputs correct.

GSIP combines these specialised responsibilities so that each corrects a weakness of the others:

LLM framing + curated playbooks + sensitivity triage + provenance gate + deterministic simulation + hybrid optimisation + uncertainty + fidelity + audit + honest explanation

The result is not a machine that is guaranteed to be right. The architecture's more defensible goal is a machine that is much less likely to hide *why it might be wrong*.

That is the project's strongest intellectual contribution: **make the system trustworthy about its own trustworthiness, refuse to silently invent the facts that matter, and only communicate a result with the confidence that the evidence and model actually support.**

Appendix A - Detailed data structures and what each field means

The architecture specification defines a set of Pydantic-style data contracts. These contracts are the shared language between services. A team member should understand them as deeply as the algorithms because architectural mistakes often arise when information is lost between stages.

A.1 Provenance enum

Provenance is not just a display label. It is an input to policy. `USER_SUPPLIED`, `MEASURED`, `RETRIEVED`, and `COMPUTED` are considered grounded classes in the initial design; `DEFAULT` is explicit but ungrounded; `LLM_RECALL` is unverified and unattributable.

An advanced implementation should consider whether `COMPUTED` deserves a single enum class or an effective-provenance field derived from parents. Keeping both `declared_provenance=COMPUTED` and `effective_trust=<weakest ancestry>` makes the lineage clearer.

A.2 ParameterValue

`name` identifies the semantic parameter. It should ideally use a stable machine identifier rather than only a human label.

`value` stores the actual quantity. For physical values, a production system should strongly consider a typed quantity with units rather than an unqualified float.

`provenance` states origin class.

`source_id` links to the concrete source artifact, instrument, API record, or dataset. A URL alone may not be enough because online data can change; immutable snapshot IDs or hashes are stronger.

`uncertainty` records error/width information. A single float is a launch simplification; mature packs may need distributions, asymmetric intervals, correlations, or confidence levels.

`parents` links a computed value to source values. This enables a provenance DAG.

`notes` carries human-readable qualifications that do not fit structured fields.

A.3 ProblemTag and ProblemClassification

A `ProblemTag` contains a problem type and calibrated confidence. The problem type should be a registry identifier, not free text, so that it maps reliably to playbooks and compatibility rules.

`ProblemClassification.tags` is ranked and multi-label.

`abstained` means no validated frame was accepted.

`generic_mode` means the system is proceeding without a validated specific frame under heightened caution. Generic mode should be visibly distinct because it changes trust expectations.

A.4 Playbook

`problem_type` connects the playbook to classification taxonomy.

`version` makes run history reproducible.

`required_factors` define the unknowns the class depends on.

`characteristic_questions` are expert-style questions and should ideally reference which factor each question can ground.

candidate_models identifies compatible domain packs/models.

sensitivity_priors seed triage before empirical screening.

known_pitfalls are structured warnings about common failure modes.

applicability_bounds declare where the reasoning frame stops being valid.

A.5 ParameterCriticality and TriageResult

ParameterCriticality.name identifies the factor.

criticality maps sensitivity into policy categories.

sensitivity stores the numerical score used to make the category.

source explains whether the score came from Morris screening or a playbook prior. A mature design can store both and the aggregation rule.

TriageResult.ranked should preserve ordering because clarification is prioritised by this ranking.

A.6 ClarificationQuestion

id allows a paused workflow to match a later answer.

parameter names the value being grounded.

question is the human-facing request.

why_needed explains decision relevance and can improve user cooperation.

produces_provenance states the expected provenance if answered. For example, a direct numerical response produces USER_SUPPLIED; a request for a dataset link can ultimately produce RETRIEVED after retrieval succeeds.

is_required allows the UI to distinguish blocking critical questions from optional refinements.

default_if_skipped makes fallback behaviour explicit rather than hidden.

A.7 DomainPackFidelity

fidelity_tier communicates model sophistication/relationship to reality.

validation_status communicates evidence status.

applicability_bounds prevent out-of-regime use.

known_limitations list omissions and simplifications.

reference gives the model's own provenance.

The convenience property is_predictive in the source design treats VALIDATED/CALIBRATED as predictive-capable. In production, predictive authority should probably also require validation status and applicability checks, not only the tier enum.

A.8 ProvenanceGateDecision

parameter says what was judged.

criticality and provenance preserve the inputs to the decision.

verdict is RUN, FLAG, or BLOCK.

action links a BLOCK/FLAG to the next workflow action, such as trigger_clarification:q3.

override records whether the policy was bypassed explicitly.

This object should always be written to the ledger because it is the evidence that the numerical core was allowed to proceed.

A.9 EnrichedState

values is a dictionary of ParameterValue rather than bare values.

subject identifies the specific asset/entity being studied when relevant.

context_summary preserves human/problem context in a controlled form.

assumptions exposes assumptions separately from measured facts.

An advanced design can split stable domain state, evidence state, and user-intent/objective state to prevent accidental mixing.

A.10 OutcomeBundle

metrics contains simulator-produced numeric outputs.

uncertainty provides per-metric bands or error information.

fidelity travels with the result so later code cannot detach a point estimate from its model quality.

A more advanced OutcomeBundle can include distributions, constraint margins, simulation diagnostics, convergence status, and references to large binary artefacts stored outside the database.

A.11 ConfidenceReport

fidelity records model credibility.

grounded_critical lists critical inputs that passed grounding.

ungrounded_critical exposes unresolved/overridden critical inputs.

overrides lists explicit bypasses.

output_bands communicates uncertainty.

verdict states whether the result can be presented as predictive, illustrative, or blocked.

The key design idea is that the confidence report is itself structured data. The LLM renders it into prose but does not decide the underlying verdict.

Appendix B - Technology choice matrix and alternatives

Concern	Launch choice	Why it fits launch	Plausible alternatives/upgrade	Main trade-off
Core language	Python 3.12	Scientific/ML/optimisation ecosystem	Rust/C++ for hot paths; polyglot workers	Python productivity vs lower-level performance
API	FastAPI	Pydantic integration, async, simple	Django/DRF, Flask, gRPC	Simplicity vs larger framework features
Contracts	Pydantic	Strong validation/JSON schemas	dataclasses + custom validation, protobuf	Developer speed vs cross-language rigidity
Database	PostgreSQL	Transactions, relational integrity, JSON	distributed SQL, document DB	Strong consistency vs specialised scaling
ORM/migrations	SQLAlchemy + Alembic	Mature Python DB tooling	SQLModel, Django ORM, raw SQL	Abstraction vs explicit SQL control
Vector retrieval	pgvector	One fewer service at launch	Milvus, Qdrant, Weaviate, Elasticsearch	Simplicity vs specialised vector scale
Workflow	Temporal	Durable pause/resume/retries	state machine + DB, Celery, Prefect	Operational weight vs workflow guarantees
Parallelism	Ray single node	Natural Python task fan-out	multiprocessing, Dask, Celery	Simplicity/scale vs operational dependency
Future parallelism	Ray cluster/cloud	Same programming model at scale	Kubernetes jobs, HPC scheduler	General cluster runtime vs domain-specific scheduler
BO	BoTorch	Modern research-grade BO on PyTorch	Optuna, scikit-optimize, Ax, custom	Flexibility vs learning curve
Evolutionary MO	pymoo / NSGA-II	Mature multi-objective operators	DEAP, Platypus, MOEA/D backends	Convenience vs backend diversity
Classifier	Prompted LLM	Fast launch, no training corpus required	Fine-tuned SLM, ensemble	Time-to-build vs control/cost
Frontend	React + TypeScript	Rich interactive state, typed client	Vue/Svelte/Angular	Ecosystem vs complexity
Packaging	Docker/Compose	Reproducible local stack	Podman, Nix, Kubernetes	Familiarity vs stronger reproducibility/ops
Pack isolation	Subprocess + limits	Lowest build cost	gVisor, Firecracker, isolated containers, WASM	Simplicity vs security strength
Run config	YAML	Human-readable, versionable	JSON/TOML	Readability vs strictness

The architectural rule is more important than any one choice: **every launch simplification should sit behind a stable interface so the upgrade can be made without redesigning the product.**

Appendix C - End-to-end pseudocode

The following pseudocode is intentionally simplified. Its purpose is to reveal the responsibilities and trust boundaries rather than prescribe exact implementation details.

GSIP Complete Technical Master Guide

```
async def run_gsip(problem_text, run_spec):
    run = ledger.create_run(problem_text, run_spec)

    classification = classify(problem_text)
    ledger.append(run, "classification", classification)

    if classification.abstained:
        return abstention_response(classification)

    playbook = registry.retrieve_playbook(classification)
    ledger.append(run, "playbook", playbook.version)

    requirements = playbook.required_factors

    triage = triage_parameters(
        requirements=requirements,
        priors=playbook.sensitivity_priors,
        cheap_simulator=selected_pack
    )
    ledger.append(run, "triage", triage)

    state = ground_known_values(problem_text, run_spec, requirements)

    while True:
        gate_decisions = [
            gate(triage.for_parameter(name), state.values[name])
            for name in requirements
        ]
        ledger.append(run, "gate", gate_decisions)

        blocking = [d for d in gate_decisions if d.verdict == BLOCK]
        if not blocking:
            break

        questions = build_triage_driven_questions(blocking)
        answers = await workflow.wait_for_user(questions)
        state = apply_grounding_answers(state, answers)

    scenarios = split_budget_generate(
        exploit=grounded_ai_proposals(state, playbook),
        explore=space_filling_samples(selected_pack.action_schema())
    )

    archive = EvaluationArchive()
    optimiser = HybridBO_NSGA2(run_spec.optimizer)

    while not optimiser.stop(archive, run_spec.budget):
        candidates = optimiser.propose(archive, scenarios)
        results = await ray_evaluate(selected_pack, state, candidates)
        archive.add(results)
        ledger.append(run, "evaluations", results)
        optimiser.update(archive)

    pareto = optimiser.pareto_front(archive)
    confidence = build_confidence_report(
        state=state,
        triage=triage,
        fidelity=selected_pack.fidelity,
        pareto=pareto
    )

    report = llm_explain(
        facts=pareto,
        confidence=confidence,
        playbook=playbook,
        hard_rules=["no invented result numbers", "propagate caveats"]
    )
```


Appendix D - Deep questions and complete answers

D.1 Why is deterministic simulation necessary but insufficient?

It is necessary because the project needs reproducible numerical evaluation independent of language-model wording. If scoring or output quantities came directly from an LLM, identical candidates could receive different answers and the system would be difficult to audit. It is insufficient because deterministic code can faithfully evaluate false inputs or an invalid model. GSIP therefore adds provenance for inputs and fidelity for models.

D.2 What is the difference between provenance and source quality?

Provenance answers "where did this value come from?" Source quality asks "how good is that source for this purpose?" A value retrieved from a named dataset has stronger provenance than an untraceable model guess, but the dataset may still be outdated, biased, mismeasured, or inapplicable. A mature gate can use both dimensions.

D.3 Why must COMPUTED inherit the weakest parent?

Because arithmetic does not create evidence. If a derived number depends on one ungrounded value, the result still depends on that weakness. Without weakest-parent inheritance, an LLM guess could be multiplied, averaged, or converted and then appear trustworthy simply because code touched it.

D.4 Why use multi-label classification?

Real problems often combine multiple frames. A generator design problem can simultaneously be electromechanical, thermal, cost-constrained, and reliability-sensitive. Selecting one label can discard variables or objectives that determine the real decision. Multi-label classification preserves competing structures and enables explicit disambiguation.

D.5 What does calibration change operationally?

Calibration makes confidence thresholds meaningful. If a classifier's raw "0.9" behaves like 0.6 in reality, a policy that proceeds whenever confidence exceeds 0.8 will be unsafe. Temperature scaling or isotonic calibration uses held-out data to better align predicted confidence with empirical correctness.

D.6 Why is abstention a feature?

Because a domain-general interface will inevitably receive requests outside validated coverage. Forcing every request into a known class produces confident misframing. Abstention protects competence boundaries and lets the system ask for clarification, switch to a clearly marked generic mode, or refuse.

D.7 What is the difference between a playbook and a domain pack?

A playbook is a reusable reasoning/framing artefact: what factors matter, what questions to ask, which models are applicable, what pitfalls exist. A domain pack is executable code: state/action schemas, simulation, scoring, feasibility, cost model, fidelity. One playbook can route to several packs with different fidelity or organisational context.

D.8 Why does triage happen before expensive grounding?

Because truth acquisition can be expensive in money, time, and user effort. If twenty variables are unknown but only three materially affect the decision, obtaining laboratory-grade evidence for all twenty is wasteful. Triage focuses grounding on the variables that can change the recommendation.

D.9 What exactly does Morris screening tell us?

Morris perturbs one input at a time across multiple points in the feasible space and measures elementary effects. Large mean absolute effects indicate influential variables; large variation suggests nonlinear or interaction behaviour. It is a screening technique, not a perfect variance decomposition.

D.10 Why is Sobol analysis more expensive?

Sobol indices estimate contributions to output variance, including interaction terms. Accurate estimation generally requires many samples and therefore many simulator calls. This can be prohibitive when each simulation is costly. Morris provides a cheaper first pass.

D.11 Why does CRITICAL + LLM_RECALL produce BLOCK?

Because an important unverified value can dominate the result. Allowing it silently would recreate v1's core failure: correct code operating on fabricated premises. The user can still run a hypothetical through explicit override, but the result becomes visibly caveated.

D.12 Why is DEFAULT treated differently from LLM_RECALL?

A default is an explicit, documented modelling assumption supplied by the pack. It can have a known basis and uncertainty. LLM recall is generally unattributable and may be invented. Both are weaker than case-specific grounded evidence, but the default is transparent and reproducible.

D.13 Why do AI-proposed scenarios still need gating?

Because a scenario can introduce new numerical facts. The fact that a value appears inside a candidate design rather than the base state does not change its origin. If the scenario relies on an invented critical location, intensity, specification, or constraint, it can still invalidate the result.

D.14 Why is model fidelity necessary if the simulator is deterministic?

Determinism says the code is repeatable; fidelity says what the code represents. A toy equation can be perfectly deterministic and still omit important physics. Fidelity prevents the natural-language layer from turning model precision into unjustified real-world authority.

D.15 What is verification versus validation versus calibration?

Verification asks whether the software correctly implements/solves the intended equations. Validation asks whether those equations/models represent reality accurately enough for intended use. Calibration adjusts parameters to match observations. A model can be verified but not validated, calibrated but overfit, or validated only in a narrow applicability regime.

D.16 How does Bayesian optimisation save simulations?

It uses results already obtained to fit a surrogate predicting performance and uncertainty. An acquisition function selects future candidates that are expected to be informative or high-value. Therefore evaluations are concentrated rather than spread blindly across the full space.

D.17 Why does surrogate uncertainty matter?

If BO used only predicted mean, it would repeatedly exploit currently promising regions and could miss better areas. Uncertainty creates a reason to explore. A candidate with moderate predicted performance but high uncertainty may be worth evaluating because it could reveal a new optimum.

D.18 What is Pareto dominance?

A solution A dominates B if A is at least as good as B on every objective and strictly better on at least one. A Pareto front contains solutions not dominated by any other evaluated solution. These represent meaningful trade-offs.

D.19 Why does NSGA-II need crowding distance?

If selection were based only on Pareto rank, many candidates could cluster in one small region of the front. Crowding distance rewards isolated points and therefore preserves diverse choices across the trade-off surface.

D.20 Why are BO and NSGA-II complementary?

BO is data-efficient but can be difficult in irregular, mixed, or many-objective spaces. NSGA-II is flexible and naturally diverse but can consume many evaluations. Sharing an archive lets BO focus expensive samples while NSGA-II maintains broad Pareto exploration.

D.21 Why is a shared evaluation archive essential?

Simulation calls are the expensive resource. Every valid evaluation should teach every search component. A shared archive avoids duplicate simulations, gives BO more training data, gives NSGA-II access to BO-discovered candidates, and centralises reproducibility/audit records.

D.22 Why does Ray fit this workload?

Candidate simulations are usually independent. Ray provides a convenient Python model for distributing these tasks across cores or machines and collecting results asynchronously. This is a natural fit for expensive black-box evaluation.

D.23 Why is durable orchestration necessary?

A provenance gate can pause for human evidence. The system must survive service restarts and resume after long waits without losing state. A durable workflow engine records progress and coordinates retries, activities, and human interaction.

D.24 Why append-only audit records?

Overwriting current state hides history. Append-only events preserve what the system believed before a clarification or override and show exactly how the state changed. This is crucial for forensic review and scientific reproducibility.

D.25 Why pgvector first rather than Milvus?

Because Postgres is already required and pgvector can support early semantic retrieval without operating another distributed database. If vector scale later becomes a bottleneck, a retrieval interface allows migration to Milvus without changing the rest of the system.

D.26 Why sandbox packs?

Third-party simulation code is arbitrary code. Without isolation it can read files, access secrets, consume resources, or attack the host. Sandboxing is therefore a security boundary, not a convenience feature.

D.27 Why must the UI expose provenance/fidelity?

If the backend carefully records uncertainty and then the UI shows only "Best option: X" in confident language, the trust design has been defeated. Users need visible evidence of what was grounded, what was assumed, what was overridden, and how strong the underlying model is.

D.28 Which parts are generic and which are domain-specific?

Generic: workflow, provenance, gate, classification contract, playbook machinery, scenario machinery, simulation runner, optimiser, ledger, reporting framework, pack SDK.

Domain-specific: equations/models, state/action schemas, feasibility rules, scoring definitions, model fidelity evidence, domain-specific playbook content, source integrations.

D.29 What does "trustworthy about its own trustworthiness" mean?

The system does not claim that provenance and optimisation make every answer correct. Instead, it explicitly represents the conditions that justify or weaken the answer. It can refuse, caveat, or label an output illustrative. The trust claim is about honesty and enforceable boundaries, not omniscience.

D.30 When should GSIP refuse predictive language?

At minimum when the pack is toy/unvalidated, a critical input remains ungrounded or was overridden, applicability bounds are violated, the classifier has no validated frame, simulation diagnostics failed, or uncertainty is too large for the claimed precision. The exact policy can be domain-specific but must be deterministic and logged.

Appendix E - Implementation checklist for engineers

E.1 Before coding

- Inventory every repository, PDF, fixture, and optimiser module.
- Produce an exists/missing/will-touch gap analysis.
- Confirm current v1 optimiser behaviour on known benchmarks.
- Identify interfaces that must remain stable.
- Do not rewrite working optimizer logic merely to match a new folder structure.

E.2 Provenance spine

- Define Provenance enum.
- Define ParameterValue with source/uncertainty/parents.
- Implement effective weakest-parent trust.
- Implement exhaustive gate matrix.
- Write gate decisions to append-only ledger.
- Add property tests for trust invariants.

E.3 Domain-pack contract

- Define schemas and model metadata.
- Require fidelity/validation/applicability/limitations/reference.
- Implement plugin discovery.
- Implement validator.
- Implement sandbox execution and resource limits.
- Create at least two deliberately unrelated reference packs to prove genericity.

E.4 Optimiser

- Preserve shared evaluation archive.
- Implement/reuse BO backend.
- Implement/reuse NSGA-II backend.
- Ensure BO can inject candidates and learn from NSGA-II evaluations.
- Ensure NSGA-II receives BO results.
- Add LHS/Sobol exploration feed.
- Add duplicate detection and deterministic candidate IDs.
- Add asynchronous Ray reconciliation.
- Carry uncertainty into Pareto representation.
- Validate on ZDT/DTLZ.

E.5 Framing and grounding

- Implement multi-label classifier contract.
- Collect calibration/OOD evaluation set.
- Implement ABSTAIN.
- Implement versioned playbook registry.
- Run Morris screening at cheap fidelity.
- Rank CRITICAL/MODERATE/LOW.
- Generate clarification questions only after triage.
- Ground via user/retrieval/measurement/computation.

- Re-run gate after each clarification.

E.6 Workflow/API/UI

- Durable run state.
- POST /runs.
- POST /runs/{id}/clarify.
- POST /runs/{id}/override.
- Resume after clarification.
- Display provenance per critical parameter.
- Display fidelity and validation status.
- Display Pareto trade-offs and uncertainty.
- Lead report with caveat when override exists.

E.7 Security and operations

- Isolate untrusted packs.
- Restrict network and secrets.
- Enforce CPU/memory/time limits.
- Log pack version/image digest.
- Add metrics/traces for workflow and simulator health.
- Add backup/retention policy for ledger and evaluation archives.
- Add authentication/authorisation before multi-user deployment.

Appendix F - A compact mental model to remember forever

Ask five questions in order:

1. **What problem is this?** - classifier + playbook.
2. **What actually matters?** - triage + sensitivity.
3. **Do we know those important things?** - grounding + provenance gate.
4. **What decisions perform best?** - deterministic simulation + BO + NSGA-II.
5. **How strongly are we entitled to state the result?** - uncertainty + fidelity + confidence report.

Everything else in the software stack exists to make those five questions reliable, scalable, secure, reproducible, and usable.

Source note: Project-specific claims are derived from the two supplied GSIP documents. Advanced mathematical and systems sections expand those concepts using standard technical knowledge and foundational references listed in the guide.